

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP

**XÂY DỰNG HỆ THỐNG PHÁT HIỆN
KHUYẾT ĐIỂM BẢNG MẠCH IN PCB SỬ
DỤNG JETSON NANO**

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

Sinh viên: **PHAN THANH THẢO**

MSSV:22139062

Hướng dẫn **ThS. TRƯƠNG QUANG PHÚC**

TP. HỒ CHÍ MINH – 6/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP

**XÂY DỰNG HỆ THỐNG PHÁT HIỆN
KHUYẾT ĐIỂM BẢNG MẠCH IN PCB SỬ
DỤNG JETSON NANO**

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

Sinh viên: **PHAN THANH THẢO**

MSSV:22139062

Hướng dẫn **ThS. TRƯƠNG QUANG PHÚC**

TP. HỒ CHÍ MINH – 6/2026

THÔNG TIN ĐỒ ÁN TỐT NGHIỆP

1. Thông tin sinh viên

- Họ và tên sinh viên: Phan Thanh Thảo
- Mã số sinh viên: 22139062
- Lớp: 22139C
- Ngành: Hệ thống nhúng và IoT
- Khoa: Điện- Điện tử
- Email: 22139062@student.hcmute.edu.vn
- Số điện thoại: 0327941755

2. Thông tin đề tài

- Tên đề tài (Tiếng Việt): Xây dựng hệ thống phát hiện khuyết điểm bảng mạch in pcb sử dụng jetson nano
- Tên đề tài (Tiếng Anh): Developing a PCB defect detection system using Jetson Nano
- Loại hình:
 - ☐ Nghiên cứu
 - ☒ Thiết kế
 - ☐ Chế tạo
 - ☐ Mô phỏng
 - ☐ Khác:
- Thời gian thực hiện:
 - Ngày giao đề tài:/...../.....
 - Ngày hoàn thành:/...../.....

3. Giảng viên hướng dẫn

- Họ và tên: Trương Quang Phúc
- Học hàm/Học vị: Thạc sĩ
- Đơn vị công tác: Khoa Điện-Điện tử

- Email: phuctq@hcmute.edu.vn

5. Tóm tắt nội dung đồ án

Nghiên cứu này đề xuất phát triển một hệ thống phát hiện khuyết điểm bảng mạch in PCB sử dụng NVIDIA Jetson Nano. Trong bối cảnh công nghiệp 4.0, việc quan sát kiểm tra ngoại quan không còn đáp ứng được yêu cầu về tốc độ và độ chính xác của các dây chuyền vừa và lớn. Hệ thống tận dụng sức mạnh của GPU và mô hình học sâu để nhận diện nhanh chóng các lỗi phổ biến như hở mạch, ngắn mạch,... Giải pháp này không chỉ tối ưu quá trình kiểm soát chất lượng mà còn giúp các doanh nghiệp điện tử giảm thiểu chi phí vận hành, loại bỏ sai sót do yếu tố con người và đẩy mạnh xu hướng nhà máy thông minh hiện nay.

7. Kết quả đạt được

- ☒ Báo cáo đồ án tốt nghiệp.
- ☒ Mô hình / Thiết bị / Phần mềm hoàn chỉnh.
- ☒ Mã nguồn chương trình.
- ☒ Kết quả mô phỏng và thực nghiệm.
- ☐ Bài báo khoa học (nếu có).
- ☐ Sản phẩm khác:

8. Kiểm tra đạo văn

- Phần mềm kiểm tra: Turnitin
- Ngày kiểm tra: 26/06/2026
- Tỷ lệ tương đồng (Similarity Index): 29 %
- Kết luận:
 - ☒ Đạt yêu cầu theo quy định của Nhà trường.
 - ☐ Không đạt yêu cầu theo quy định của Nhà trường.

9. Cam kết của sinh viên

Tôi cam kết rằng đồ án tốt nghiệp này là kết quả nghiên cứu và thực hiện của riêng tôi dưới sự hướng dẫn của giảng viên hướng dẫn. Các tài liệu tham khảo, số liệu, hình ảnh và kết quả nghiên cứu của người khác đều được trích dẫn đầy đủ theo đúng quy định. Tôi hoàn toàn chịu trách nhiệm trước Nhà trường về tính trung thực và tính hợp pháp của nội dung đồ án.

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: Lấy mẫu hệ thống phát hiện khuyết điểm bằng mạch in PCB sử dụng Teton Nano

Sinh viên: + Phạm Thanh Thảo MSSV: 22131062

+ MSSV:

Hướng dẫn: Th.S. Trương Quang Phúc

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

hợp lý, mục tiêu rõ ràng

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

phương pháp hợp lý, rõ ràng

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá/ kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

KQ, có phân tích

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

kết luận hợp lý

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

trích dẫn T.T.T

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

Đáp ứng yêu cầu nhà trường

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Bảo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Đạt yêu cầu

Tp. HCM, ngày 23 tháng 6 năm 2026

Người nhận xét
(Ký và ghi rõ họ tên)

Trương Quang Phúc

* Giảng viên hướng dẫn có thể ghi tiếp vào mặt sau

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành và sâu sắc đến Thầy **Trương Quang Phúc**, người đã tận tình hướng dẫn, định hướng và hỗ trợ em trong suốt quá trình học tập cũng như thực hiện đề tài Đồ án tốt nghiệp. Chính sự tâm huyết và niềm đam mê sâu sắc của Thầy đối với lĩnh vực chuyên môn đã truyền cảm hứng mạnh mẽ, giúp em thêm vững tin và quyết tâm hoàn thành tốt nhiệm vụ được giao.

Chính sự tâm huyết, tận tụy và niềm đam mê sâu sắc của Thầy đối với lĩnh vực kỹ thuật đã truyền cho em nguồn cảm hứng to lớn. Qua từng buổi làm việc và trao đổi, em không chỉ tiếp thu được kiến thức chuyên môn mà còn học hỏi được tinh thần nghiên cứu nghiêm túc và thái độ làm việc chuyên nghiệp những điều mà em tin rằng sẽ theo em suốt chặng đường học tập và làm việc sau này.

Trong quá trình thực hiện đồ án, em đã cố gắng nỗ lực hết mình, vận dụng những kiến thức đã được học kết hợp với việc tự tìm hiểu tài liệu thực tế nhằm hoàn thành đề tài một cách tốt nhất. Tuy nhiên, do thời gian nghiên cứu có hạn cùng với kinh nghiệm thực hành còn chưa sâu, em hiểu rằng báo cáo này vẫn còn tồn tại một số thiếu sót nhất định. Em rất mong nhận được sự thông cảm và những ý kiến đóng góp quý báu từ Thầy để em có thể tiếp tục hoàn thiện và nâng cao chất lượng của đề tài.

Một lần nữa, em xin gửi lời cảm ơn chân thành đến Thầy. Em kính chúc Thầy luôn dồi dào sức khỏe, hạnh phúc và tiếp tục truyền cảm hứng cho nhiều thế hệ sinh viên tiếp theo trên con đường học tập và nghiên cứu khoa học.

LỜI CAM ĐOAN

Em cam đoan đề tài này là kết quả nghiên cứu độc lập dưới sự hướng dẫn của thầy Trương Quang Phúc. Toàn bộ ý tưởng, thiết kế phần cứng và mã nguồn đều do em tự thực hiện, không sao chép dưới mọi hình thức. Tài liệu tham khảo chỉ dùng để hỗ trợ lý thuyết. Em xin chịu hoàn toàn trách nhiệm và chấp nhận mọi hình thức kỷ luật từ Nhà trường nếu phát hiện có gian lận hay vi phạm bản quyền.

Sinh viên cam đoan

Phan Thanh Thảo

TÓM TẮT

Nghiên cứu này đề xuất phát triển một hệ thống phát hiện khuyết điểm bảng mạch in PCB sử dụng NVIDIA Jetson Nano. Trong bối cảnh công nghiệp 4.0, việc quan sát kiểm tra ngoại quan không còn đáp ứng được yêu cầu về tốc độ và độ chính xác của các dây chuyền vừa và lớn. Hệ thống tận dụng sức mạnh của GPU và mô hình học sâu để nhận diện nhanh chóng các lỗi phổ biến như hở mạch, ngắn mạch,... Giải pháp này không chỉ tối ưu quá trình kiểm soát chất lượng mà còn giúp các doanh nghiệp điện tử giảm thiểu chi phí vận hành, loại bỏ sai sót do yếu tố con người và đẩy mạnh xu hướng nhà máy thông minh hiện nay.

MỤC LỤC

DANH MỤC HÌNH ẢNH.....	vii
DANH MỤC BẢNG.....	ix
CÁC TỪ VIẾT TẮT.....	x
CHƯƠNG 1 TỔNG QUAN	1
1.1. GIỚI THIỆU	1
1.2. TÌNH HÌNH NGUYÊN CỨU	2
1.3. MỤC TIÊU ĐỀ TÀI	2
1.4. PHƯƠNG PHÁP NGUYÊN CỨU	4
1.5. BỐ CỤC ĐỀ TÀI	4
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT.....	5
2.1. TỔNG QUAN VỀ PHÁT HIỆN ĐỐI TƯỢNG VÀ BÀI TOÁN NHẬN DIỆN LỖI PCB	5
2.1.1. Khái niệm về phát hiện đối tượng (Object Detection):	5
2.1.2. Tổng quan về bài toán nhận diện lỗi PCB	5
2.2. TỔNG QUAN MÔ HÌNH YOLOV10	7
2.2.1. Khái niệm YOLOv10	7
2.2.2. Cơ chế hoạt động của YOLOv10	8
2.2.3. Các mô hình biến thể của YOLOv10	10
2.3. TỔNG QUAN VỀ MÁY TÍNH NHÚNG NVIDIA JETSON NANO .	12
2.3.1. Thông số kỹ thuật của máy tính nhúng NVIDIA Jetson Nano	12
2.3.2. Ứng dụng thực tiễn của NVIDIA Jetson Nano	14
2.4 TỔNG QUAN VỀ DOCKER	16
2.4.1. Khái niệm Docker và lý do sử dụng	16
2.4.2. Lợi ích và ứng dụng của Docker	17
2.5 TỔNG QUAN VỀ 8MP IMX179 USB CAMERA MODULE	18
2.5.1 Tổng quan về camera.....	18

2.5.2 Thông số kỹ thuật của camera	19
2.5.3 Ứng dụng thực tiễn	20
CHƯƠNG 3 THIẾT KẾ HỆ THỐNG	21
3.1. PHÂN TÍCH YÊU CẦU HỆ THỐNG.....	21
3.1.1. Yêu cầu chức năng	21
3.1.2 Yêu cầu phi chức năng.....	22
3.1.3. Yêu cầu phần cứng và phần mềm.....	22
3.1.4 Lựa chọn phần cứng dựa trên yêu cầu.....	23
3.2. LỰA CHỌN VÀ THIẾT KẾ MÔ HÌNH.....	24
3.2.1. Lựa chọn mô hình trên cơ sở lý thuyết	24
3.2.1. So sánh các phiên bản YOLO trên dữ liệu huấn luyện	27
3.2.2. Lý do chọn Yolov10s.....	30
3.2.3. Quy trình huấn luyện mô hình.....	32
3.3. KIẾN TRÚC TỔNG QUAN CỦA HỆ THỐNG.....	37
3.4. XÂY DỰNG GIAO DIỆN NGƯỜI DÙNG VÀ TRIỂN KHAI TRÊN DOCKER	39
3.4.1. Lựa chọn công nghệ	40
3.4.2. Xây dựng giao diện người dùng.....	40
3.4.3. Thiết kế các thành phần UI	43
3.4.4. Triển khai ứng dụng.....	44
3.4.5 Hướng dẫn sử dụng hệ thống.....	46
3.5 THIẾT KẾ MÔ HÌNH 3D	47
3.5.1 Yêu cầu về thiết kế mô hình 3D	47
3.5.2 Phân tích thiết kế mô hình 3D.....	48
3.5.3 Kết quả thiết kế mô hình 3D	49
CHƯƠNG 4 KẾT QUẢ VÀ THẢO LUẬN	53
4.1. KẾT QUẢ THIẾT KẾ HỆ THỐNG.....	53

4.2. ĐÁNH GIÁ MÔ HÌNH.....	56
4.2.1. Đánh giá độ chính xác theo từng loại lỗi.....	56
4.2.2 Đánh giá hiệu năng xử lý theo thời gian thực.....	57
4.2.3. Đánh giá tính ổn định và độ tin cậy vận hành.....	58
4.2.4. So sánh kết quả với mục tiêu thiết kế ban đầu.....	59
CHƯƠNG 5 TÓM LƯỢC VÀ KẾT LUẬN.....	60
5.1 TÓM LƯỢC DỰ ÁN.....	60
5.2 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	60
5.2.1 Kết luận	60
5.2.2 Hạn chế và hướng phát triển	61
TÀI LIỆU THAM KHẢO	65

DANH MỤC HÌNH ẢNH

Hình 1.1 Các lỗi mạch in phổ biến.....	3
Hình 2.1. Hiệu năng của Yolov10 so với các mô hình trước [8].....	7
Hình 2.2. Consistent dual assignment cho huấn luyện không NMS.....	9
Hình 2.3. Kết quả so sánh các mô hình Yolov10	11
Hình 2.4. Bo phát triển NVIDIA Jetson Nano	12
Hình 2.5. Hình ảnh ứng dụng thực tế.....	15
Hình 2.6. Logo của docker	16
Hình 2.7. Hình ảnh Camera Module	19
Hình 2.8. Các ứng dụng của Camera	20
Hình 3.1 So sánh Yolov10 và các mô hình	26
Hình 3.2. Một số mẫu PCB trong tập dữ liệu	33
Hình 3.3. Ảnh trong tập dữ liệu sau khi qua xử lý.....	33
Hình 3.4. Thông số huấn luyện mô hình.....	35
Hình 3.5. Confusion matrix normalized.....	36
Hình 3.6. Kết quả tổng quát của mô hình	37
Hình 3.7. Sơ đồ khối tổng quát	39
Hình 3.8. Sơ đồ hoạt động của chương trình	41
Hình 3.9. Giao diện ứng dụng	44
Hình 3.10. Hình ảnh tổng quan thiết kế	49
Hình 3.11. Vị trí đặt camera	50
Hình 3.12. Mô hình thực tế	51
Hình 3.13. Hình ảnh thực tế vị trí camera.....	52
Hình 3.14. Ảnh chụp mạch in	53
Hình 4.1. Sản phẩm thực tế.....	54
Hình 4.2. Ảnh PCB thông qua camera.....	55
Hình 4.3. Kết quả mạch thử nghiệm 1	55

Hình 4.4. Kết quả mạch thử nghiệm 2	55
Hình 4.5. Kết quả mạch thử nghiệm 3	56

DANH MỤC BẢNG

Bảng 2.1. Thông số kỹ thuật của NVIDIA Jetson Nano	12
Bảng 2.2 Thông số kỹ thuật Camera.....	19
Bảng 3.1. Đặc tính cấu tạo của mô hình	27
Bảng 3.2. Kết quả đo lường độ chính xác	27
Bảng 3.3. Tốc độ của các mô hình	29
Bảng 3.4. Tốc độ mô hình khi giảm kích thước đầu vào	30
Bảng 3.5. Nhận xét kết quả Confusion Matrix Normalized.....	36
Bảng 4.1. Kết quả đánh giá độ chính xác theo từng loại lỗi	57
Bảng 4.2. Thời gian xử lý trung bình theo độ phân giải đầu vào trên Jetson Nano	57
Bảng 4.3. So sánh mục tiêu thiết kế và kết quả thực nghiệm	59

CÁC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Nghĩa tiếng Việt
PCB	Printed Circuit Board	Bảng mạch in
YOLO	You Only Look Once	Thuật toán phát hiện đối tượng một giai đoạn
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
AI	Artificial Intelligence	Trí tuệ nhân tạo
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
CPU	Central Processing Unit	Bộ xử lý trung tâm
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
IoT	Internet of Things	Internet vạn vật
AOI	Automated Optical Inspection	Kiểm tra quang học tự động
NMS	Non-Maximum Suppression	Thuật toán loại bỏ hộp dự đoán trùng lặp
mAP	mean Average Precision	Độ chính xác trung bình
AP	Average Precision	Độ chính xác trung bình
IoU	Intersection over Union	Chỉ số đo độ chồng lấp giữa hai bounding box

FLOPs	Floating Point Operations	Số phép toán dấu phẩy động
UI	User Interface	Giao diện người dùng
cuDNN	CUDA Deep Neural Network	Thư viện tăng tốc mạng nơ-ron sâu
Docker	Docker Container Platform	Nền tảng đóng gói và triển khai ứng dụng
CI/CD	Continuous Integration / Continuous Deployment	Tích hợp liên tục / triển khai liên tục
USB	Universal Serial Bus	Chuẩn kết nối USB
HDMI	High-Definition Multimedia Interface	Cổng xuất hình ảnh độ nét cao
CSI	Camera Serial Interface	Giao tiếp camera nối tiếp
GPIO	General Purpose Input/Output	Chân vào/ra đa dụng
UVC	USB Video Class	Chuẩn camera USB
FPS	Frames Per Second	Số khung hình trên giây

CHƯƠNG 1 TỔNG QUAN

1.1. GIỚI THIỆU

Thế giới đang bước vào một giai đoạn chuyển mình mạnh mẽ với cuộc Cách mạng Công nghiệp lần thứ tư (Công nghiệp 4.0). Ở đó, ranh giới giữa thế giới thực và ảo trở nên mong manh hơn bao giờ hết nhờ vào sự hội tụ của các công nghệ số tiên tiến. Các công nghệ kỹ thuật số, vốn đã có những bước phát triển vượt bậc trong vài thập kỷ gần đây, nay được đặt trên một nền tảng mới với sức mạnh được nhân lên gấp bội. Sự phổ cập của Internet tốc độ cao, sự ra đời của các chuẩn kết nối không dây thế hệ mới (như 5G, Wi-Fi 6), cùng với những tiến bộ đột phá trong hiệu năng phần cứng điện tử và khả năng xử lý, lưu trữ dữ liệu khổng lồ (Big Data) đã tạo ra một cơn địa chấn, làm thay đổi toàn diện bộ mặt của ngành sản xuất công nghiệp [1] [2].

Sự kết hợp hài hòa giữa phần cứng thông minh và phần mềm tối ưu đã kiến tạo nên một nền tảng vững chắc cho quá trình số hóa và tự động hóa toàn diện trong mọi khâu của hoạt động sản xuất. Điều này không chỉ dừng lại ở việc cải tiến dây chuyền lắp ráp mà còn tác động sâu sắc đến cách thức các doanh nghiệp quản lý, vận hành và tối ưu hóa hiệu suất của từng thiết bị. Trong bối cảnh đó, ngành sản xuất thiết bị điện tử đóng vai trò là xương sống của nền kinh tế số. Trái tim của mọi thiết bị hiện đại, từ điện thoại thông minh đến các hệ thống điều khiển công nghiệp phức tạp, chính là bảng mạch in (PCB). Sự bùng nổ của các thiết bị IoT và hệ thống nhúng đòi hỏi các tấm PCB ngày càng có mật độ linh kiện cao hơn, kích thước nhỏ gọn hơn và cấu trúc đa lớp phức tạp. Điều này đặt ra một thách thức vô cùng lớn cho quy trình kiểm tra chất lượng tại các nhà máy.

Phương pháp kiểm tra ngoại quan thủ công truyền thống bằng mắt người đang dần bộc lộ những hạn chế cố hữu: tốc độ xử lý chậm, dễ bị ảnh hưởng bởi sự mệt mỏi của người lao động và thiếu tính khách quan, dẫn đến tỉ lệ sai sót (lọt lỗi) đáng kể. Để giải quyết bài toán này, các hệ thống kiểm tra quang học tự động (AOI) đã ra đời. Tuy nhiên, các hệ thống AOI truyền thống dựa trên thuật toán xử lý ảnh

cổ điển thường gặp khó khăn trong việc thích nghi với các mẫu lỗi biến dạng phức tạp hoặc điều kiện ánh sáng thay đổi, gây ra tỉ lệ "báo động giả" cao [3].

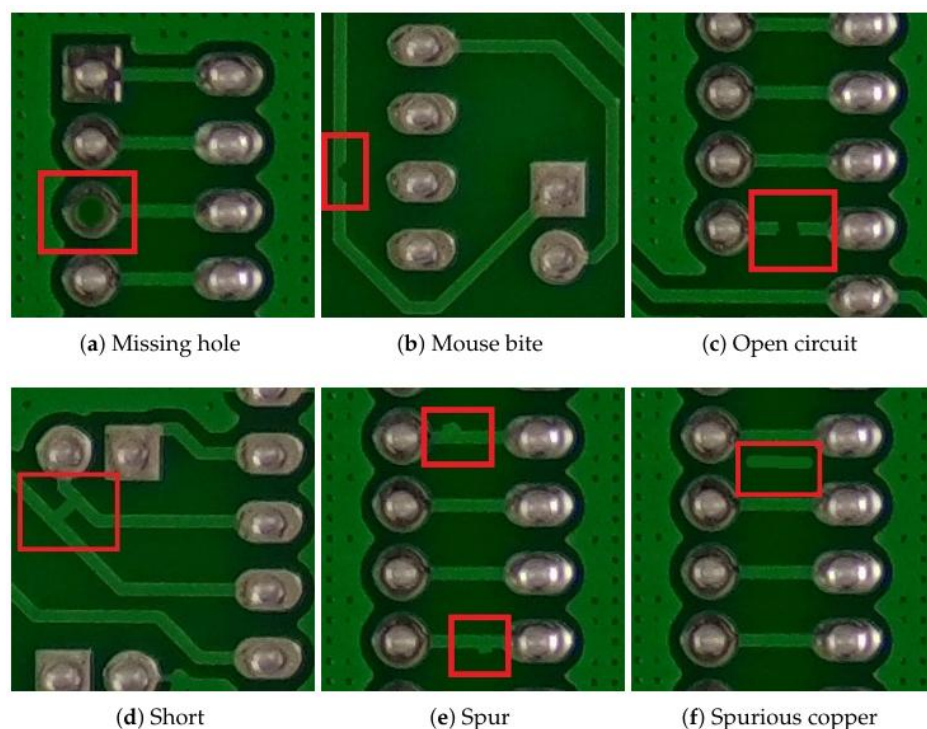
Sự trỗi dậy của trí tuệ nhân tạo, đặc biệt là học sâu (Deep Learning), đã mở ra một chương mới cho lĩnh vực kiểm tra khuyết điểm PCB. Việc tích hợp các mô hình nhận diện đối tượng tiên tiến lên các nền tảng điện toán biên (Edge Computing) như NVIDIA Jetson Nano cho phép hệ thống thực hiện phân tích dữ liệu hình ảnh ngay tại dây chuyền sản xuất với độ trễ cực thấp [4]. Giải pháp này không chỉ đảm bảo độ chính xác vượt trội mà còn mang lại khả năng triển khai linh hoạt, chi phí đầu tư tối ưu, phù hợp với mô hình nhà máy thông minh mà cuộc Cách mạng Công nghiệp 4.0 đang hướng tới.

1.2. TÌNH HÌNH NGUYÊN CỨU

Việc kiểm tra lỗi bảng mạch in (PCB) đã trải qua nhiều giai đoạn phát triển, từ quan sát thủ công đến các hệ thống tự động hóa sử dụng thị giác máy tính. Trên thế giới, các hệ thống kiểm tra quang học tự động (AOI) truyền thống thường dựa trên phương pháp so sánh hình ảnh (Image Comparison) hoặc kiểm tra dựa trên quy tắc (Rule-based). Tuy nhiên, các phương pháp này bộc lộ hạn chế khi môi trường ánh sáng thay đổi hoặc với các bảng mạch có mật độ linh kiện cao. Tại Việt Nam, lĩnh vực kiểm tra lỗi sản phẩm bằng AI đang nhận được sự quan tâm lớn từ cả giới hàn lâm lẫn doanh nghiệp [5]. Nhiều nhóm nghiên cứu tại các trường đại học kỹ thuật lớn đã triển khai các đề tài nhận diện lỗi linh kiện, phân loại rác thải hoặc kiểm tra sản phẩm may mặc bằng xử lý ảnh. Tuy nhiên, các nghiên cứu chuyên sâu về lỗi bề mặt PCB (như lỗi đường đồng, lỗi chân hàn) trên các hệ thống nhúng thực tế vẫn còn đang trong giai đoạn phát triển và hoàn thiện. Do đó, nhu cầu nghiên cứu về áp dụng các phần cứng biên với chi phí thấp cùng với các mô hình trí tuệ nhân tạo mới để phát hiện lỗi bề mặt PCB nói riêng và các vấn đề khác trên toàn thế giới nói chung góp phần xây dựng, cải tiến, đưa các công nghệ mới áp dụng vào công nghiệp và đời sống.

1.3. MỤC TIÊU ĐỀ TÀI

Công nghệ ngày càng phát triển dẫn đến kích thước linh kiện ngày càng nhỏ, mật độ linh kiện tăng và tính chất phức tạp, đa dạng của quá trình sản xuất PCB, PCB trở nên dễ bị tác động bởi nhiều yếu tố, bao gồm ma sát cơ học, nhiễu tĩnh điện và ăn mòn hóa học trong quá trình sản xuất. Những yếu tố này có thể dẫn đến nhiều loại khuyết tật chẳng hạn như mouse bit (lỗ đục biên dạng), spur (gai đồng), missing hole (thiếu lỗ khoan), short (ngắn mạch), open circuit (hở mạch), spurious copper (đồng thừa) [6]. Các đặc trưng của lỗi sẽ được thể hiện ở hình bên dưới.



Hình 1.1 Các lỗi mạch in phổ biến

Theo như bài khảo sát [6] cho thấy các lỗi trên là những lỗi phổ biến ở mạch PCB. Từ đó tôi quyết định sẽ xây dựng mô hình phát hiện 6 lỗi này trên mạch in PCB.

Quy mô của mô hình sẽ phù hợp với các doanh nghiệp vừa và nhỏ nên thiết kế sẽ có phần bán thủ công. Hệ thống sẽ cần người dùng thủ công đưa mô hình vào buồng tối để chụp ảnh PCB. Kích thước mạch PCB tối đa được chọn là 10x10cm. Để đáp ứng đủ về mặt chính xác, tin cậy và thời gian với một hệ thống

thủ công thì mức thời gian thiết kế tôi đưa ra thấp hơn 1 giây cho hệ thống phát hiện lỗi của PCB. Ngoài ra về giao diện người dùng phải thiết kế dễ thao thác, dễ dàng triển khai, lỗi PCB sẽ thể hiện trực tiếp trên giao diện.

1.4. PHƯƠNG PHÁP NGUYÊN CỨU

Để triển khai đề tài dựa trên một phương pháp luận khoa học, kết hợp giữa nguyên cứu và lý thuyết và thực nghiệm, được thực thực hiện qua các bước sau:

Bước đầu tiên là nguyên cứu lý thuyết. Ở bước này các công việc được thực hiện như sau:

- Tìm hiểu và so sánh các mô hình nhận diện đối tượng hiện có trên Internet.
- Tìm kiếm và tổng hợp các nguồn dữ liệu (ảnh) lỗi PCB.

Bước kế tiếp là thiết kế hệ thống. Các bước thực hiện như sau:

- Xây dựng và huấn luyện mô hình dựa trên nguồn dữ liệu đã thu thập trên nền tảng Google Colab.
- Triển khai mô hình trên phần cứng NVIDIA Jetson Nano.
- Xây dựng giao diện người dùng.

Bước cuối cùng là triển khai hệ thống. Các bước thực hiện ở bước cuối cùng như sau:

- Thiết kế mô hình để trưng bày sản phẩm và chạy thực nghiệm
- Kiểm tra và đánh giá mô hình trong môi trường thực tế.

1.5. BỐ CỤC ĐỀ TÀI

Chương 1 Giới thiệu tổng quan về đề tài, tình hình nguyên cứu trong và ngoài nước, phương pháp nguyên cứu và mục tiêu của đề tài.

Chương 2 Trình bày các lý thuyết về mô hình YoloV10s, phần cứng Jetson Nano, Docker và các công cụ xây dựng hệ thống khác.

Chương 3 Trình bày các yêu cầu thiết kế của hệ thống, sơ đồ kết nối, sơ đồ khối của phần mềm, quá trình thiết kế giao diện và hướng dẫn sử dụng.

Chương 4 Trình bày kết quả của hệ thống đã được xây dựng và các chức năng đi kèm.

Chương 5 Nhận xét về nền tảng đã thiết kế, ưu và nhược điểm và hướng phát triển trong tương lai

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT

2.1. TỔNG QUAN VỀ PHÁT HIỆN ĐỐI TƯỢNG VÀ BÀI TOÁN NHẬN DIỆN LỖI PCB

2.1.1. Khái niệm về phát hiện đối tượng (Object Detection):

Phát hiện đối tượng(Object Detection) là kỹ thuật trong Computer Vison dùng để xác định vị trí và phân loại nhiều đối tượng trong hình ảnh hoặc video. Không chỉ nhận biết có đối tượng gì, mà còn xác định vị trí (tọa độ hộp bao quanh (bounding box)) của chúng [7]. Ví dụ: Hệ thống camera có thể nhận diện được trong khung hình đối tượng nào là xe hơi, xe đạp, con người.

Mục tiêu cốt lõi của việc áp dụng phát hiện đối tượng (Object Detection) là cho phép máy tính có khả năng nhận biết và định vị chính xác các thực thể trong môi trường thực tế thông qua dữ liệu hình ảnh. Kỹ thuật này đóng vai trò then chốt trong việc vận hành các hệ thống tự động hóa hiện đại, từ xe tự lái, giám sát an ninh thông minh cho đến quy trình kiểm định chất lượng sản phẩm công nghiệp. Bằng cách cung cấp những dữ liệu trực quan về vị trí và loại đối tượng, công nghệ này không chỉ giúp tăng cường độ chính xác trong phân tích hình ảnh mà còn hỗ trợ hệ thống đưa ra các quyết định xử lý kịp thời và hiệu quả hơn.

2.1.2. Tổng quan về bài toán nhận diện lỗi PCB

Trong quá trình sản xuất phơi mạch in, các sai sót về hóa chất hoặc cơ khí có thể dẫn đến những khiếm khuyết trên bề mặt đồng và các lỗ khoan. Dựa trên đặc điểm hình học và tính chất điện, các lỗi phổ biến bao gồm:

- Vết cắn (Mouse bite): Là hiện tượng một phần đường mạch đồng bị khuyết mất (như bị gặm), làm giảm tiết diện dẫn điện, dễ gây quá nhiệt hoặc đứt mạch tại điểm đó.
- Vết lồi (Spur): Một phần đồng dư thừa nhô ra từ đường mạch nhưng chưa chạm vào đường mạch kế bên. Đây là mầm mống có thể dẫn đến chạm mạch sau này.
- Thiếu lỗ khoan (Missing hole): Các lỗ via hoặc lỗ xuyên tâm không được khoan đúng vị trí hoặc bị bít kín, khiến việc kết nối tín hiệu giữa các lớp hoặc lắp linh kiện không thể thực hiện.
- Chạm mạch (Short): Hiện tượng dư thừa đồng kết nối hai đường tín hiệu vốn dĩ phải cách điện với nhau, gây đoản mạch và hỏng hệ thống khi cấp điện.
- Hở mạch (Open circuit): Đường mạch đồng bị đứt hoàn toàn tại một điểm, khiến dòng điện không thể lưu thông giữa hai đầu kết nối.
- Đồng dư (Spurious copper): Những mảng đồng thừa nằm rải rác trên bề mặt bo mạch ở những vị trí không có đường mạch. Chúng có thể gây nhiễu điện từ hoặc tạo ra các kết nối ngoài ý muốn.

Sự hiện diện của các khiếm khuyết trên bo mạch in gây ra "hiệu ứng domino", ảnh hưởng trực tiếp đến ba khía cạnh cốt lõi:

Thứ nhất là suy giảm hiệu suất điện và độ tin cậy của sản phẩm. Các lỗi như Short hoặc Open circuit làm cho bo mạch hoàn toàn không hoạt động ngay từ khâu kiểm thử đầu tiên. Ngoài ra còn hỏng hóc tiềm ẩn (Latent Defects) là tác động nguy hiểm nhất từ các lỗi như Mouse bite hoặc Spur. Một vết khuyết nhỏ có thể chưa làm hỏng mạch ngay lập tức, nhưng dưới tác động của nhiệt độ và dòng điện trong quá trình sử dụng, vị trí đó sẽ trở thành "điểm nóng" (hotspot), dẫn đến đứt mạch hoặc cháy nổ sau một thời gian ngắn tới tay khách hàng.

Thứ hai, các khuyết điểm trên bo mạch còn gây lãng phí chi phí sản xuất. Trong trường hợp bo mạch bị lỗi không được phát hiện doanh nghiệp sẽ tiếp tục đưa những bo mạch hỏng này vào dây chuyền lắp ráp linh kiện. Khi đó, các linh kiện

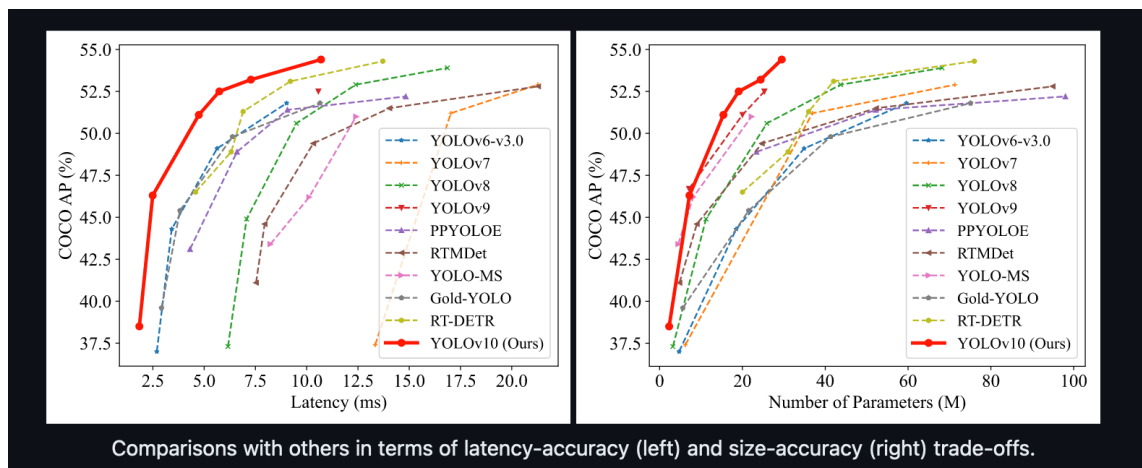
đắt tiền như CPU, RAM, chipset sẽ được hàn lên một tấm mạch hỏng, gây lãng phí toàn bộ vật tư. Việc loại bỏ bo mạch lỗi càng muộn, giá trị thiệt hại càng cao. Việc nhận diện lỗi ngay từ giai đoạn phôi mạch giúp loại bỏ hoặc sửa chữa kịp thời, giảm thiểu tối đa tỷ lệ sản phẩm hỏng cuối dây chuyền.

Cuối cùng, nó còn gây gián đoạn dây chuyền và uy tín thương hiệu. Việc phát hiện lỗi thủ công bằng mắt thường chậm và dễ sai sót. Nếu hệ thống kiểm định tự động không hoạt động tốt, nó sẽ làm chậm toàn bộ tốc độ xuất xưởng. Ngoài ra, nếu các lỗi nhỏ lọt qua khâu kiểm soát và đến tay người dùng, doanh nghiệp sẽ phải đối mặt với chi phí bảo hành khổng lồ và sụt giảm lòng tin từ khách hàng.

2.2. TỔNG QUAN MÔ HÌNH YOLOV10

2.2.1. Khái niệm Yolov10

Yolov10 là một kiến trúc thị giác máy tính tiên tiến được thiết kế để phát hiện đối tượng theo thời gian thực, xây dựng dựa trên những tiến bộ của các phiên bản trước. Mô hình Yolov10 đạt được trung bình Average Precision (mAP) cao hơn các mô hình Yolo trước [8].



Hình 2.1. Hiệu năng của Yolov10 so với các mô hình trước [8]

Từ hình ảnh trên, ta dễ dàng nhận thấy Yolov10 đạt được hiệu suất tổng thể vượt trội về cả độ trễ và số lượng tham số ở dải kích thước thấp và trung bình so

với nhiều mô hình phát hiện đối tượng khác. Yolov10 thể hiện sự cân bằng tối ưu giữa độ chính xác, độ trễ và kích thước tham số.

Điểm khác biệt cốt lõi của YOLOv10 nằm ở chiến lược thiết kế tổng thể hướng đến tối ưu hóa hiệu suất - hiệu quả (efficiency-accuracy driven design strategy) . Các cải tiến chính bao gồm [8]:

- Đầu phân loại nhẹ (Lightweight classification head): Giảm dư thừa tính toán.
- Hạ mẫu phân tách không gian-kênh (Spatial-channel decoupled downsampling): Tối ưu hóa trích xuất các đặc trưng.
- Thiết kế khối dựa trên xếp hạng (Rank-guided block design): Tinh gọn kiến trúc.
- Tích hợp tích chập hạt nhân lớn (Large-kernel convolution) và cơ chế tự chú ý một phần (Partial self-attention): Nâng cao độ chính xác với chi phí tính toán tối thiểu.

2.2.2. Cơ chế hoạt động của Yolov10

2.2.2.1. Chiến lược huấn luyện không NMS với gán nhãn kép (Dual Label Assignments)

Các mô hình YOLO truyền thống sử dụng chiến lược gán một-nhiều (one-to-many) trong huấn luyện, gán nhiều mẫu dương cho một đối tượng thực tế. Cách này cung cấp tín hiệu giám sát dồi dào nhưng yêu cầu sử dụng NMS trong quá trình suy luận, gây ra độ trễ [8].

YOLOv10 khắc phục bằng cách kết hợp cả hai chiến lược gán nhãn:

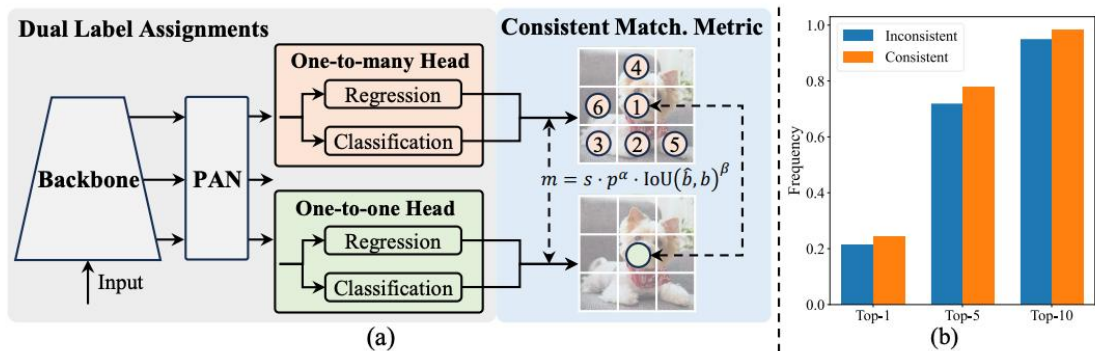
- Gán một-một (One-to-One): Gán một dự đoán duy nhất cho mỗi đối tượng thực tế. Giúp loại bỏ nhu cầu sử dụng NMS, cho phép triển khai đầu cuối (end-to-end) nhưng thường có giám sát yếu hơn.

- Gán một-nhiều (One-to-Many): Cung cấp tín hiệu giám sát mạnh mẽ nhưng yêu cầu NMS khi suy luận.

YOLOv10 bổ sung thêm một đầu một-một (one-to-one head) song song với nhánh một-nhiều ban đầu. Trong quá trình huấn luyện, cả hai đầu được tối ưu đồng thời, tận dụng nguồn giám sát phong phú từ nhánh một-nhiều. Khi suy luận, chỉ đầu một-một được sử dụng, do đó loại bỏ hoàn toàn NMS, đạt hiệu suất cao mà không tăng chi phí.

2.2.2.2. Chỉ số đối sánh nhất quán (Consistent Matching Metric)

Một thành phần quan trọng là chỉ số đối sánh nhất quán được áp dụng cho cả hai nhánh. Chỉ số này kết hợp giữa điểm phân loại (classification score) và Intersection over Union (IoU) giữa hộp giới hạn dự đoán và thực tế. Việc sử dụng cùng một chỉ số cho cả hai nhánh đảm bảo sự giám sát hài hòa, từ đó nâng cao chất lượng dự đoán.



Hình 2.2. Consistent dual assignment cho huấn luyện không NMS

2.2.2.3. Tối ưu hóa thành phần kiến trúc

YOLOv10 tập trung tối ưu ba thành phần chính: các lớp hạ mẫu (downsampling layers), các giai đoạn và đầu ra [9].

Đầu phân loại nhẹ: Phân tích cho thấy đầu hồi quy (regression head) quan trọng hơn đầu phân loại. YOLOv10 sử dụng kiến trúc nhẹ cho đầu phân loại với các tích chập depthwise separable, giảm đáng kể FLOPs và tham số mà không ảnh hưởng nhiều đến hiệu suất.

Hạ mẫu phân tách không gian - kênh: Thay vì sử dụng một tích chập 3×3 stride bằng 2 để đồng thời hạ mẫu không gian và biến đổi kênh, YOLOv10 tách thành hai bước: tích chập pointwise để điều chỉnh số kênh, tiếp theo là tích chập depthwise để hạ mẫu không gian. Phương pháp này giảm chi phí tính toán, số lượng tham số và giữ lại nhiều thông tin hơn.

Thiết kế khối dựa trên xếp hạng: YOLOv10 sử dụng phân tích hạng nội tại (intrinsic rank analysis) để xác định sự dư thừa. Dựa trên kết quả, mô hình giới thiệu khối đảo ngược nhỏ gọn (Compact Inverted Block - CIB) và chiến lược phân bổ khối dựa trên xếp hạng, thay thế các giai đoạn dư thừa nhất bằng thiết kế tinh gọn hơn mà không làm giảm hiệu suất.

2.2.2.4. Tích chập hạt nhân lớn và cơ chế tự chú ý

Tích chập hạt nhân lớn (Large-Kernel Convolution): Được áp dụng có chiến lược ở các giai đoạn sâu, sử dụng hạt nhân kích thước 7×7 để mở rộng vùng tiếp nhận (receptive field) mà không ảnh hưởng đến các đặc trưng nông (cần cho vật thể nhỏ) và không làm tăng độ trễ.

Cơ chế tự chú ý một phần (Partial Self-Attention - PSA): Để tận dụng khả năng mô hình hóa toàn cục của cơ chế tự chú ý nhưng giảm độ phức tạp tính toán, YOLOv10 đưa ra khối PSA. Khối này chỉ áp dụng cơ chế tự chú ý trên một nửa số kênh đặc trưng, nửa còn lại giữ nguyên, sau đó kết hợp lại. PSA được đặt ở giai đoạn có độ phân giải thấp nhất để giảm thiểu chi phí.

2.2.3. Các mô hình biến thể của Yolov10

YOLOv10 cung cấp nhiều biến thể mô hình, được thiết kế để phù hợp với từng yêu cầu về tài nguyên và hiệu suất [9]:

- YOLOv10n (Nano) – Tối ưu cho các thiết bị có giới hạn nghiêm ngặt về bộ nhớ và năng lực tính toán.
- YOLOv10s (Small) – Lựa chọn cân bằng giữa tốc độ xử lý và độ chính xác, phù hợp với các ứng dụng thời gian thực thông thường.
- YOLOv10m (Medium) – Biến thể đa dụng, đáp ứng tốt hầu hết các bài toán phát hiện đối tượng với chi phí hợp lý.
- YOLOv10b (Balanced) – Tăng độ rộng mạng so với phiên bản Medium, mang lại độ chính xác cao hơn mà vẫn giữ được sự ổn định.
- YOLOv10l (Large) – Ưu tiên độ chính xác, phù hợp khi có sẵn tài nguyên tính toán mạnh hơn.
- YOLOv10x (Extra Large) – Biến thể lớn nhất, hướng đến hiệu suất tối đa, lý tưởng cho các hệ thống yêu cầu độ chính xác cao nhất.

Hiệu suất của các mô hình được đo lường trên tập dữ liệu COCO với TensorRT FP16 trên T4 GPU được kết quả như sau [9]:

Model	Input Size	AP ^{val}	FLOPs (G)	Latency (ms)
YOLOv10n	640	38.5	6.7	1.84
YOLOv10s	640	46.3	21.6	2.49
YOLOv10m	640	51.1	59.1	4.74
YOLOv10b	640	52.5	92.0	5.74
YOLOv10l	640	53.2	120.3	7.28
YOLOv10x	640	54.4	160.4	10.70

Hình 2.3. Kết quả so sánh các mô hình YOLOv10

Từ ảnh trên ta có thể thấy ở YOLOv10n mặc dù điểm số AP(Average Precision) trên tập dữ liệu kiểm định thấp nhất trong số các mô hình nhưng

FLOPs(Floating Point Operations) thấp nhất trong các mô hình. Nó thấp hơn xấp xỉ 24 lần so với Yolov10x(mô hình đạt độ chính xác cao nhất trong ảnh) về FLOPs và còn về độ trễ nó thấp hơn gấp 5 lần so với Yolov10x. Các mô hình phân cấp bậc và mục đích sử dụng giống với đã nêu trên.

2.3. TỔNG QUAN VỀ MÁY TÍNH NHÚNG NVIDIA JETSON NANO

NVIDIA Jetson Nano là một máy tính nhúng (Edge Computing) mạnh mẽ, được thiết kế chuyên dụng để chạy các ứng dụng trí tuệ nhân tạo (AI) tại biên. Với kích thước nhỏ gọn và mức tiêu thụ điện năng thấp, đây là lựa chọn tối ưu để tích hợp các mô hình trí tuệ nhân tạo ứng dụng vào thực tiễn.



Hình 2.4. Bo phát triển NVIDIA Jetson Nano

2.3.1. Thông số kỹ thuật của máy tính nhúng NVIDIA Jetson Nano

Bảng 2.1. Thông số kỹ thuật của NVIDIA Jetson Nano

CPU	64-bit Quad-core ARM A57 @1.43Ghz
GPU	128 core NVIDIA Maxwell @921Mhz
Memory	4GB 64 bit LPDDR4 @ 1600MHz 25.6 GB/s
Video Encoder	4Kp30 (4x) 1080p30 (2x) 1080p60

Video Decoder	4Kp60 (2x) 4Kp30 (8x) 1080p30 (4x) 1080p60
USB	4x USB 3.0 USB 2.0 Micro B
Camera	MIPI CSI-2 x 2
Display	HDMI DisplayPort
Networking	Gigabit Ethernet (RJ45)
Wireless	M.2 Key E with PCIe x1
Storage	MicroSD card
Other I/O	(3x) I2C (2x) SPI UART I2S GPIOs

Trái tim của Jetson Nano là bộ vi xử lý trung tâm CPU Quad-core ARM A57 hoạt động tại xung nhịp 1.43GHz. Cấu trúc 64-bit này cho phép thiết bị xử lý mượt mà các tác vụ hệ điều hành và các thuật toán logic phức tạp. Tuy nhiên, điểm tạo nên sự khác biệt thực sự nằm ở bộ vi xử lý đồ họa GPU NVIDIA Maxwell với 128 nhân CUDA. Kiến trúc Maxwell cho phép Jetson Nano chạy các thư viện AI hiện đại như TensorFlow, PyTorch và Caffe một cách hiệu quả. Với sức mạnh này, thiết bị có khả năng thực hiện đồng thời nhiều mạng thần kinh nhân tạo khác nhau cho các ứng dụng như phân loại hình ảnh, phát hiện đối tượng, phân đoạn ngữ nghĩa và xử lý giọng nói.

Để đáp ứng lưu lượng dữ liệu khổng lồ trong các tác vụ Deep Learning, Jetson Nano được trang bị bộ nhớ 4GB LPDDR4 64-bit với băng thông lên tới 25.6 GB/s. Tốc độ truy xuất dữ liệu nhanh chóng này giúp giảm thiểu độ trễ khi nạp các mô hình AI lớn vào RAM. Khả năng xử lý video của thiết bị cũng cực kỳ ấn tượng, đóng vai trò quan trọng trong các hệ thống giám sát thông minh. Jetson Nano hỗ trợ giải mã video (Video Decoder) lên tới độ phân giải 4Kp60 và mã hóa video (Video Encoder) ở mức 4Kp30. Điều này cho phép thiết bị xử lý đồng thời nhiều luồng video chất lượng cao (ví dụ: 8 luồng 1080p30 cùng lúc), biến nó thành một "trạm" xử lý thị giác máy tính thực thụ.

Về mặt vật lý và giao tiếp, Jetson Nano cung cấp một hệ sinh thái cổng kết nối đa dạng, tương đương với một máy tính cá nhân thu nhỏ:

Kết nối dữ liệu: Với 4 cổng USB 3.0, người dùng có thể dễ dàng kết nối với các cảm biến, máy ảnh USB hoặc ổ cứng ngoài. Cổng Gigabit Ethernet đảm bảo tốc độ truyền tải dữ liệu mạng ổn định, cần thiết cho việc truyền dẫn dữ liệu lên đám mây hoặc giao tiếp trong mạng nội bộ.

Thị giác máy tính: Thiết bị tích hợp 2 cổng MIPI CSI-2, cho phép kết nối trực tiếp với các mô-đun camera chất lượng cao, giúp tối ưu hóa luồng dữ liệu hình ảnh đầu vào cho AI mà không làm tiêu tốn quá nhiều tài nguyên CPU.

Mở rộng và Lưu trữ: Mặc dù sử dụng thẻ MicroSD làm phương thức lưu trữ chính để tiết kiệm không gian, Jetson Nano vẫn cung cấp khe cắm M.2 Key E cho phép mở rộng các mô-đun không dây (Wi-Fi/Bluetooth) qua giao thức PCIe.

Giao tiếp công nghiệp: Các chân I/O khác như I2C, SPI, UART, I2S và GPIOs giúp Jetson Nano dễ dàng tương tác với các loại cảm biến, bộ điều khiển động cơ và các vi điều khiển khác (như Arduino hay STM32), tạo nên một hệ thống nhúng hoàn chỉnh cho robot tự hành hoặc thiết bị IoT thông minh.

Tóm lại, NVIDIA Jetson Nano là sự kết hợp hoàn hảo giữa hiệu suất tính toán GPU cao cấp và tính linh hoạt của hệ thống nhúng. Với khả năng hỗ trợ đầy đủ các chuẩn hiển thị hiện đại như HDMI và DisplayPort, cùng với khả năng tiêu thụ điện năng thấp, đây là lựa chọn hàng đầu cho các kỹ sư và nhà phát triển đang tìm kiếm một giải pháp hiện thực hóa các dự án AI thực tế, từ robot tự vận hành đến các hệ thống nhận diện khuôn mặt và phân tích dữ liệu thời gian thực tại biên.

2.3.2. Ứng dụng thực tiễn của NVIDIA Jetson Nano

Khả năng xử lý song song nhờ các nhân CUDA cho phép Jetson Nano giải quyết các bài toán mà những máy tính nhúng truyền thống (như Raspberry Pi) gặp khó khăn. Các ứng dụng của nó tập trung vào việc phân tích dữ liệu ngay tại thiết bị mà không cần gửi về máy chủ đám mây, giúp giảm độ trễ và tăng tính bảo mật.

Một trong những ứng dụng phổ biến nhất là thị giác máy tính và giám sát thông minh. Nhờ bộ mã hóa/giải mã video 4K chuyên dụng từ đó thiết bị có thể Nhận diện khuôn mặt, đếm số lượng người, phát hiện hành vi bất thường hoặc theo dõi đối tượng trong thời gian thực. Một ví dụ thực tiễn trong đời sống là hệ thống giám sát Tuân thủ & An toàn Công nghiệp: Giải pháp Phân tích Video dựa trên AI để Quản lý Vận hành Sân bay [10]. Thiết bị sẽ xử lý các kênh video cục bộ để thực hiện các tác vụ như đếm hành khách, theo dõi tỷ lệ chuyển đổi và đo lưu lượng giao thông thành phố.



Hình 2.5. Hình ảnh ứng dụng thực tế

Ngoài ra NVIDIA Jetson Nano còn góp mặt vào nhiều lĩnh vực như:

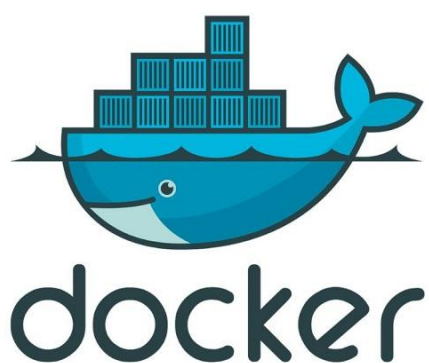
- Trong môi trường nông nghiệp, nơi kết nối internet thường không ổn định, khả năng xử lý tại chỗ của Jetson Nano trở nên vô cùng giá trị. Thiết bị đóng góp phân tích tình trạng sức khỏe cây trồng, phát hiện sâu bệnh và tự động hóa thu hoạch.
- Y tế và Chăm sóc Sức khỏe nhờ phân tích hình ảnh y tế nhanh (X-quang, MRI) hoặc hỗ trợ thiết bị đeo thông minh.

- Robot tự hành và Xe tự lái. Nhờ các giao tiếp I/O đa dạng như PWM, I2C và UART, Jetson Nano có thể điều khiển trực tiếp các động cơ servo và đọc dữ liệu từ cảm biến LiDAR hoặc IMU. Từ đó lập bản đồ (SLAM), tránh vật cản và lập kế hoạch đường đi dựa trên học sâu.

2.4 TỔNG QUAN VỀ DOCKER

2.4.1. Khái niệm Docker và lý do sử dụng

Trong quy trình phát triển và vận hành phần mềm truyền thống, thách thức lớn nhất thường nằm ở sự thiếu đồng nhất giữa các môi trường thực thi. Việc thiết lập thủ công từ cài đặt công cụ, cấu hình thư viện đến quản lý các biến môi trường trên nhiều máy chủ không chỉ gây tốn kém về nguồn lực mà còn dẫn đến những sai lệch tiềm ẩn giữa máy tính của lập trình viên và môi trường thực tế (Production). Chính sự khác biệt về phiên bản hệ điều hành hay các phụ thuộc phần mềm đã tạo ra rào cản lớn cho việc triển khai nhanh. Docker ra đời như một cuộc cách mạng hóa nhằm tiêu chuẩn hóa quy trình này, loại bỏ hoàn toàn sự phụ thuộc vào hạ tầng vật lý bên dưới để đảm bảo rằng ứng dụng có thể vận hành ổn định trong một "chính thể" duy nhất dù được đặt ở bất kỳ đâu.



Hình 2.6. Logo của docker

Docker được định nghĩa là một nền tảng ảo hóa ở cấp độ hệ điều hành, cho phép các kỹ sư đóng gói toàn bộ ứng dụng và các thành phần liên quan vào trong một đơn vị phần mềm độc lập được gọi là Container. Khác với các phương pháp ảo hóa truyền thống vốn đòi hỏi một hệ điều hành khách riêng biệt cho mỗi ứng dụng, Docker tận dụng trực tiếp nhân của hệ điều hành máy chủ để khởi tạo các môi trường cô lập tuyệt đối [11].

Cơ chế này không chỉ giúp tối ưu hóa hiệu suất tài nguyên mà còn mang lại tính khả chuyển vượt trội. Khi một ứng dụng được đóng gói thành một Docker Image, nó trở thành một bản thiết kế bất biến chứa đựng mọi thứ cần thiết để thực thi. Việc triển khai lúc này chỉ đơn giản là khởi chạy các Container từ những Image đó trên bất kỳ hệ thống nào có hỗ trợ Docker Engine. Nhờ vậy, quá trình từ phát triển đến vận hành được kết nối liền mạch, cho phép ứng dụng khởi động tức thì và hoạt động nhất quán mà không cần quan tâm đến các cấu hình phức tạp của máy chủ đích [11].

2.4.2. Lợi ích và ứng dụng của Docker

Việc áp dụng Docker mang lại những giá trị vượt trội nhờ khả năng chuẩn hóa toàn bộ vòng đời của ứng dụng. Lợi ích đầu tiên và quan trọng nhất chính là tối ưu hóa hiệu suất tài nguyên. Khác với ảo hóa truyền thống (Virtual Machines) vốn tiêu tốn một lượng lớn tài nguyên cho hệ điều hành khách, Docker tận dụng cơ chế chia sẻ nhân hệ điều hành máy chủ, cho phép hàng chục hoặc hàng trăm container chạy song song mà không làm suy giảm hiệu năng hệ thống. Điều này không chỉ giúp tiết kiệm chi phí hạ tầng mà còn đẩy nhanh tốc độ khởi động ứng dụng lên mức gần như tức thì, tạo điều kiện cho việc mở rộng quy mô (scaling) một cách linh hoạt theo lưu lượng truy cập thực tế [11].

Bên cạnh đó, Docker thiết lập một quy trình CI/CD (Lập trình và Triển khai liên tục) nhất quán. Nhờ tính chất "đóng gói một lần, chạy mọi nơi", đội ngũ phát triển có thể tự tin rằng mã nguồn hoạt động trên máy cá nhân sẽ vận hành hoàn hảo trên môi trường kiểm thử và máy chủ thực tế. Sự cách ly tuyệt đối giữa các

container cũng đảm bảo tính bảo mật và ổn định, ngăn chặn tình trạng xung đột giữa các dịch vụ khác nhau chạy trên cùng một máy chủ vật lý, từ đó giảm thiểu tối đa các lỗi hệ thống phát sinh do môi trường.

Về mặt ứng dụng thực tiễn, Docker đóng vai trò là nền tảng cốt lõi cho kiến trúc Microservices. Thay vì xây dựng một khối phần mềm khổng lồ và khó bảo trì, các nhà phát triển chia nhỏ hệ thống thành các dịch vụ độc lập, mỗi dịch vụ được đặt trong một container riêng biệt. Cách tiếp cận này cho phép mỗi thành phần của hệ thống (như cơ sở dữ liệu, bộ nhớ đệm hay giao diện người dùng) có thể được cập nhật, thay thế hoặc vá lỗi mà không gây ảnh hưởng đến toàn bộ cấu trúc chung.

Ngoài ra, Docker còn được ứng dụng rộng rãi trong việc đơn giản hóa quản lý cơ sở dữ liệu và môi trường lập trình. Thay vì phải cài đặt phức tạp các phiên bản SQL, Python hay Node.js khác nhau trực tiếp lên máy tính, lập trình viên chỉ cần kích hoạt các container tương ứng để có ngay một môi trường làm việc sạch sẽ và đúng chuẩn. Trong kỷ nguyên điện toán đám mây (Cloud Computing), Docker kết hợp cùng các công cụ điều phối như Kubernetes đã trở thành tiêu chuẩn công nghiệp, giúp các doanh nghiệp vận hành hệ thống phần mềm với độ tin cậy và khả năng tự động hóa ở mức cao nhất.

2.5 TỔNG QUAN VỀ 8MP IMX179 USB CAMERA MODULE

2.5.1 Tổng quan về camera

Camera là module camera USB 2.0 được thiết kế nhỏ gọn, sử dụng cảm biến Sony IMX179 độ phân giải 8 megapixel. Sản phẩm nổi bật với góc nhìn siêu rộng (lên đến 150° đường chéo), tích hợp ống kính M12, cho phép lấy nét từ 0.5m đến vô cực. Camera hỗ trợ hai định dạng dữ liệu MJPG và YUY2, tương thích với nhiều hệ điều hành phổ biến nhờ chuẩn UVC (USB Video Class), dễ dàng sử dụng mà không cần cài đặt driver phức tạp.



Hình 2.7. Hình ảnh Camera Module

2.5.2 Thông số kỹ thuật của camera

Bảng 2.2 Thông số kỹ thuật Camera

Thông số	Chi tiết
Cảm biến	Sony IMX179 CMOS, kích thước 1/3.2 inch, điểm ảnh 1.4 μm , độ phân giải tối đa 8 MP (3264x2448)
Ống kính	Vùng nhìn thấy 75°
Định dạng & tốc độ khung hình	MJPEG: 15 fps @ 8MP, 30 fps @ 1080P; YUY2: 30 fps @ 640x480
Điện & môi trường	Nguồn 5V DC, dòng tối đa 350mA, nhiệt độ hoạt động -20°C đến +75°C, cáp dài 1m (tùy chọn 2/3/5m)
Tương thích	Windows, Linux, Mac, Android (chuẩn UVC), hỗ trợ micro đơn (tùy chọn hai kênh)

Camera UB0239 nổi bật với góc nhìn rộng 75° kết hợp cảm biến Sony IMX179 8MP, phù hợp cho các ứng dụng cần bao phủ không gian lớn. Tuy nhiên, ở độ phân giải tối đa, tốc độ khung hình chỉ đạt 15fps (MJPEG), hạn chế cho các tác vụ yêu cầu chuyển động mượt mà. Điểm đáng chú ý là dải nhiệt độ hoạt động rộng (-20°C đến 75°C) và tiêu thụ dòng điện khá thấp (350mA), giúp thiết bị ổn định trong nhiều môi trường khắc nghiệt.

2.5.3 Ứng dụng thực tiễn

Nhờ góc nhìn siêu rộng và độ phân giải cao, camera rất thích hợp cho các hệ thống giám sát không gian lớn như siêu thị, nhà xưởng hoặc văn phòng. Trong lĩnh vực hội nghị từ xa và giáo dục trực tuyến, thiết bị có thể ghi hình toàn bộ phòng họp hoặc bảng giảng mà không cần xoay ngang. Ngoài ra, sản phẩm còn được sử dụng trong thị giác máy, nhận diện khuôn mặt, phát trực tiếp (live stream) và các hệ thống nhúng nhờ khả năng hoạt động ổn định trên nhiều nền tảng và không cần cài driver phức tạp.



security monitoring



industrial equipment



self-service terminals



security gates



security monitoring



advertising machines



ATM machines



more equipment applications

Hình 2.8. Các ứng dụng của Camera

CHƯƠNG 3 THIẾT KẾ HỆ THỐNG

3.1. PHÂN TÍCH YÊU CẦU HỆ THỐNG

3.1.1. Yêu cầu chức năng

Yêu cầu chức năng xác định các nhiệm vụ cụ thể mà hệ thống phải thực hiện, phản ánh trực tiếp hành vi của thiết bị trong quá trình vận hành. Đối với hệ thống kiểm tra khuyết điểm PCB, các yêu cầu này được xây dựng nhằm đảm bảo quy trình kiểm tra diễn ra liên tục, chính xác và ít phụ thuộc vào tác động của con người.

Ở phần mục tiêu đề tài, quy mô của mô hình sẽ phù hợp với các doanh nghiệp vừa và nhỏ. Yêu cầu đầu tiên, hệ thống phải có khả năng thu nhận hình ảnh thời gian thực từ camera với độ trễ thấp, xử lý hình ảnh đúng thời gian đã đặt ra. Nguồn hình ảnh có thể đến từ camera CSI hoặc USB, nhưng yêu cầu nhất quán là tín hiệu phải ổn định và tương thích với bộ xử trung tâm để phục vụ cho các tác vụ suy luận phía sau.

Tiếp theo, hệ thống cần nhận diện và phân loại các loại lỗi phổ biến trên PCB, bao gồm short circuit, open circuit, missing hole, spur... Quá trình này được thực hiện bởi mô hình học sâu đã được huấn luyện trước, với khả năng phát hiện đồng thời nhiều lỗi trên cùng một bo mạch. Kết quả nhận diện không chỉ dừng ở việc phát hiện có lỗi hay không, mà còn phải xác định được vị trí chính xác của khuyết điểm trên không gian ảnh, phục vụ cho công đoạn đánh dấu hoặc phân loại sản phẩm sau đó.

Để người vận hành có thể theo dõi và can thiệp kịp thời, giao diện hiển thị trực quan kết quả và cảnh báo lỗi đóng vai trò then chốt. Hệ thống phải cung cấp một màn hình trực tiếp hiển thị luồng video kèm theo bounding box và nhãn lỗi tương ứng. Khi phát hiện khuyết điểm, hệ thống tự động kích hoạt cảnh báo trực quan. Tất cả thông tin về lỗi, thời điểm phát hiện và hình ảnh lưu trữ đều phải được ghi nhận để phục vụ cho việc đánh giá chất lượng sau này.

3.1.2 Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng mang tính tác nghiệp, hệ thống kiểm tra khuyết điểm PCB cần đáp ứng một số yêu cầu phi chức năng then chốt, quyết định đến khả năng ứng dụng thực tế cũng như độ tin cậy khi vận hành trong môi trường sản xuất công nghiệp.

Độ chính xác là yêu cầu hàng đầu, bởi bất kỳ sai sót nào trong khâu kiểm tra đều có thể dẫn đến chi phí sửa chữa tăng cao hoặc sản phẩm lỗi lọt ra thị trường. Hệ thống cần đảm bảo tỷ lệ nhận diện sai ở mức thấp nhất có thể, đặc biệt đối với các khuyết điểm nghiêm trọng như chập mạch (short circuit) hay hở mạch (open circuit). Đồng thời, tỷ lệ bỏ sót cũng phải được kiểm soát chặt chẽ, tránh tình trạng sản phẩm mạch bị lỗi nhưng vẫn được đánh giá là đạt. Các chỉ số này không chỉ phụ thuộc vào mô hình học sâu mà còn liên quan đến chất lượng dữ liệu huấn luyện, quy trình tiền xử lý và khả năng điều chỉnh ngưỡng tin cậy phù hợp với yêu cầu của dây chuyền sản xuất.

Tốc độ xử lý là yếu tố sống còn để hệ thống có thể vận hành đồng bộ với nhịp độ sản xuất thực tế đảm bảo không gây ứ đọng sản phẩm trên băng chuyền. Điều này đòi hỏi việc lựa chọn mô hình phải cân bằng giữa độ chính xác và chi phí tính toán.

Tính đóng gói và dễ dàng triển khai thể hiện qua khả năng quản lý môi trường phụ thuộc một cách nhất quán. Hệ thống sẽ được triển khai trên nhiều thiết bị khác nhau trong nhà máy, do đó cần tránh xung đột thư viện và đảm bảo tái lập môi trường giống hệt từ môi trường phát triển.

3.1.3. Yêu cầu phần cứng và phần mềm

Để đảm bảo hệ thống nhận diện khuyết điểm PCB hoạt động ổn định, chính xác và đáp ứng được các ràng buộc về hiệu năng cũng như tính sẵn sàng triển khai, việc lựa chọn nền tảng phần cứng và phần mềm là yếu tố nền tảng. Trong phạm vi thiết kế này, tôi tập trung vào giải pháp nhúng nhẹ nhưng vẫn đủ mạnh để vận hành mô hình học sâu theo thời gian thực.

Về phần cứng: Do hệ thống vận hành theo cơ chế bán tự động, thiết bị phần cứng được lựa chọn phải đảm bảo tốc độ xử lý tối thiểu 1 ảnh/giây và hoạt động ổn định trong thời gian dài. Hệ thống cần tích hợp sẵn hệ điều hành để thuận tiện cho việc nâng cấp, khắc phục sự cố, đồng thời sở hữu mức chi phí tối ưu, phù hợp với năng lực tài chính của các doanh nghiệp vừa và nhỏ (SMEs). Ngoài ra, thiết bị phải hỗ trợ các cổng giao tiếp phổ biến (như HDMI, USB,...) để dễ dàng kết nối ngoại vi. Cuối cùng, module camera/chụp ảnh đi kèm phải có độ phân giải cao, đảm bảo chất lượng hình ảnh rõ nét và đáng tin cậy.

Về phần mềm: Hệ thống sở hữu giao diện trực quan, thân thiện với người dùng và quy trình khởi động đơn giản. Phần mềm cũng được tối ưu hóa sâu nhằm giảm tải tối đa cho hệ thống, từ đó giải phóng tài nguyên để tập trung xử lý chính xác tác vụ nhận diện lỗi.

3.1.4 Lựa chọn phần cứng dựa trên yêu cầu

Về phần cứng, trung tâm xử lý được chọn là NVIDIA Jetson Nano. Hệ thống sử dụng máy tính nhúng NVIDIA Jetson Nano làm trung tâm xử lý chính. Thiết bị sở hữu GPU tích hợp kiến trúc Maxwell, hỗ trợ tính toán song song mạnh mẽ thông qua các thư viện tăng tốc phần cứng như CUDA và TensorRT. Với kích thước nhỏ gọn, mức tiêu thụ điện năng thấp (5W - 10W) và khả năng xử lý đa luồng, Jetson Nano đặc biệt phù hợp cho các ứng dụng thị giác máy trong môi trường công nghiệp. Bên cạnh đó, với chi phí đầu tư tối ưu (khoảng 100 USD), giải pháp này hoàn toàn khả thi để triển khai tại các doanh nghiệp vừa và nhỏ (SMEs). Để thu thập dữ liệu hình ảnh, hệ thống hỗ trợ kết nối camera qua giao tiếp CSI nhằm tối ưu hóa băng thông và giảm độ trễ, hoặc qua cổng USB để tăng tính linh hoạt trong quá trình phát triển. Các linh kiện phụ trợ đi kèm bao gồm: nguồn điện 5V/4A, thẻ nhớ microSD (dung lượng tối thiểu 64GB) phục vụ lưu trữ hệ điều hành và dữ liệu vận hành, cùng màn hình hiển thị hoặc kết nối mạng để tương tác với giao diện người dùng.

Về thiết bị chụp mạch in PCB tôi chọn camera 8mp imx179 usb camera vì các tính năng ưu việt của nó. Thiết bị có lấy nét tự động (auto focus) là tính năng giúp camera tự động xác định và điều chỉnh tiêu điểm. Thay vì lấy nét thủ công,

hệ thống này sẽ đảm bảo chủ thể của ảnh sẽ luôn sắc nét, rõ ràng một cách nhanh chóng. Camera này hỗ trợ độ phân giải lên đến 3264 x 2448 giúp chụp mạch in PCB một cách rõ nét và chi tiết đưa vào mô hình nhận diện chính xác. Ngoài ra, camera hỗ trợ USB giúp dễ dàng kết nối và hỗ trợ Linux (hệ điều hành mà Jetson nano sử dụng).

Về phần mềm, hệ thống được xây dựng dựa trên nền tảng Linux Ubuntu dành riêng cho Jetson (JetPack SDK), cung cấp sẵn các driver GPU, thư viện CUDA, cuDNN và TensorRT. Tầng ứng dụng được phát triển bằng ngôn ngữ Python nhờ hệ sinh thái phong phú trong lĩnh vực học sâu. Để quản lý các thư viện phụ thuộc như PyTorch, OpenCV, Ultralytics YOLO và các thành phần giao diện, toàn bộ môi trường thực thi được đóng gói trong Docker, giúp tách biệt với hệ điều hành nền, dễ dàng triển khai, sao chép và bảo trì. Đặc biệt, Docker kết hợp với NVIDIA Container Runtime cho phép container truy cập trực tiếp vào GPU, đảm bảo hiệu năng suy luận không bị suy giảm. Nhờ sự kết hợp này, hệ thống vừa đáp ứng yêu cầu về hiệu suất xử lý thời gian thực, vừa đảm bảo tính đóng gói và nhất quán trong môi trường vận hành thực tế.

3.2. LỰA CHỌN VÀ THIẾT KẾ MÔ HÌNH

3.2.1. Lựa chọn mô hình trên cơ sở lý thuyết

Trong bối cảnh ràng buộc về mặt thời gian thực hiện đồ án tốt nghiệp, việc lựa chọn mô hình huấn luyện đòi hỏi sự cân bằng nghiêm ngặt giữa tiến độ triển khai và chất lượng đầu ra. Vì vậy, nghiên cứu này tiến hành lựa chọn và áp dụng kiến trúc mạng YOLO. Lựa chọn này được củng cố bởi hai luận điểm cốt lõi sau:

- Độ tin cậy và giá trị học thuật: YOLO là một mô hình chuẩn chuẩn hóa trong bài toán phát hiện đối tượng thời gian thực. Hiệu năng của thuật toán này đã được kiểm chứng, phản biện và khẳng định vị thế vững chắc trong hàng loạt tạp chí, hội nghị khoa học uy tín về trí tuệ nhân tạo. Việc kế thừa

một giải pháp có độ tin cậy cao giúp giảm thiểu rủi ro trong quá trình thực nghiệm và rút ngắn thời gian tối ưu hóa thuật toán.

- Tính linh hoạt và khả năng tương thích phần cứng: Hệ sinh thái kiến trúc của YOLO được thiết kế với dải cấu hình rất rộng, phân cấp rõ ràng theo quy mô tham số (parameter size) và độ phức tạp tính toán. Ưu điểm này mang lại khả năng mở rộng (scalability) linh hoạt, giúp dễ dàng định vị và triển khai phiên bản phù hợp, đáp ứng chính xác các giới hạn về băng thông bộ nhớ và năng lực xử lý của hệ thống phần cứng đã định sẵn mà không làm suy giảm hiệu suất vận hành của toàn hệ thống.

Trong tiến trình phát triển của các mô hình mạng nơ-ron đặc trưng cho bài toán phát hiện đối tượng thời gian thực, họ thuật toán YOLO liên tục ghi nhận những cải tiến vượt bậc về mặt kiến trúc, tiêu biểu là các thế hệ tiên tiến gần đây như YOLOv12 và YOLO26. Tuy nhiên, việc lựa chọn cấu hình mô hình triển khai thực tế không chỉ dựa trên tính mới, mà phải là kết quả của quá trình tối ưu hóa đa mục tiêu nhằm cân bằng nghiêm ngặt giữa độ chính xác (mAP) và độ trễ suy luận trên mỗi khung hình.

Qua quá trình nghiên cứu lý thuyết và khảo sát thực nghiệm, đề tài này tôi quyết định lựa chọn kiến trúc YOLOv10 làm giải pháp cốt lõi. Quyết định này được củng cố bởi những cải tiến của YOLOv10 trong việc loại bỏ tầng hậu xử lý NMS (Non-Maximum Suppression-free) thông qua cơ chế consistent dual assignment. Đặc tính này giúp giảm thiểu tối đa chi phí tính toán và thời gian thực thi trên từng đơn vị ảnh, mang lại một phương án hài hòa, vừa đáp ứng yêu cầu xử lý thời gian thực, vừa đảm bảo hiệu suất nhận diện ổn định trong giới hạn tài nguyên của dự án.

Theo bài báo [12], YOLOv10 đạt được những chỉ số ấn tượng so với YOLOv8 trên tập dữ liệu COCO với cùng một cài đặt huấn luyện và được train trên cùng GPU T4 với TensorRT FP16 với các biến thể N / S / M /L /X lần lượt như sau. Trên chỉ số AP YOLOv10 đạt được tăng trưởng 1,2% / 1,4% / 0.5% / 0.3% / 0.5%, với tham số ít hơn 28% / 36% / 41% / 44% / 57%, 23% / 24% / 25% / 27% / 38% ít hơn về chi phí tính toán và ấn tượng hơn về độ trễ giảm 70% / 65% / 50% / 41%

/ 37%. Ngoài ra còn các mô hình khác cũng được so sánh ở bảng sau được trích từ bài báo [12].

Model	#Param.(M)	FLOPs(G)	AP ^{val} (%)	Latency(ms)	Latency ^f (ms)
YOLOv6-3.0-N [29]	4.7	11.4	37.0	2.69	1.76
Gold-YOLO-N [60]	5.6	12.1	39.6	2.92	1.82
YOLOv8-N [21]	3.2	8.7	37.3	6.16	1.77
YOLOv10-N (Ours)	2.3	6.7	38.5 / 39.5[†]	1.84	1.79
YOLOv6-3.0-S [29]	18.5	45.3	44.3	3.42	2.35
Gold-YOLO-S [60]	21.5	46.0	45.4	3.82	2.73
YOLO-MS-XS [8]	4.5	17.4	43.4	8.23	2.80
YOLO-MS-S [8]	8.1	31.2	46.2	10.12	4.83
YOLOv8-S [21]	11.2	28.6	44.9	7.07	2.33
YOLOv9-S [65]	7.1	26.4	46.7	-	-
RT-DETR-R18 [78]	20.0	60.0	46.5	4.58	4.49
YOLOv10-S (Ours)	7.2	21.6	46.3 / 46.8[†]	2.49	2.39
YOLOv6-3.0-M [29]	34.9	85.8	49.1	5.63	4.56
Gold-YOLO-M [60]	41.3	87.5	49.8	6.38	5.45
YOLO-MS [8]	22.2	80.2	51.0	12.41	7.30
YOLOv8-M [21]	25.9	78.9	50.6	9.50	5.09
YOLOv9-M [65]	20.0	76.3	51.1	-	-
RT-DETR-R34 [78]	31.0	92.0	48.9	6.32	6.21
RT-DETR-R50m [78]	36.0	100.0	51.3	6.90	6.84
YOLOv10-M (Ours)	15.4	59.1	51.1 / 51.3[†]	4.74	4.63
YOLOv6-3.0-L [29]	59.6	150.7	51.8	9.02	7.90
Gold-YOLO-L [60]	75.1	151.7	51.8	10.65	9.78
YOLOv9-C [65]	25.3	102.1	52.5	10.57	6.13
YOLOv10-B (Ours)	19.1	92.0	52.5 / 52.7[†]	5.74	5.67
YOLOv8-L [21]	43.7	165.2	52.9	12.39	8.06
RT-DETR-R50 [78]	42.0	136.0	53.1	9.20	9.07
YOLOv10-L (Ours)	24.4	120.3	53.2 / 53.4[†]	7.28	7.21
YOLOv8-X [21]	68.2	257.8	53.9	16.86	12.83
RT-DETR-R101 [78]	76.0	259.0	54.3	13.71	13.58
YOLOv10-X (Ours)	29.5	160.4	54.4 / 54.4[†]	10.70	10.60

Hình 3.1 So sánh Yolov10 và các mô hình

Xét đến các biến thể kiến trúc thể hệ mới, tiêu biểu như YOLOv26, mô hình này được kế thừa và phát triển trực tiếp dựa trên nền tảng lý thuyết của YOLOv10 [13]. Điểm cải tiến cốt lõi từ phía các nhà nghiên cứu trong phiên bản này tập trung vào việc tái cấu trúc các toán tử và phân đoạn mạng nhằm tối ưu hóa hiệu năng suy luận chuyên biệt cho bộ vi xử lý trung tâm (CPU). Bằng cách giảm thiểu tối đa chi phí tính toán dư thừa và tinh chỉnh cơ chế lan truyền, giải pháp này giải quyết triệt để bài toán thắt nút cổ chai về mặt tài nguyên. Qua đó, mô hình cho phép triển khai mượt mà các thuật toán học sâu trên các thiết bị phổ thông hoặc hệ

thông phần cứng công nghiệp vốn bị giới hạn về năng lực xử lý đồ họa (GPU). Nhưng mà để cân bằng giữa tốc độ và độ chính xác tôi vẫn chọn YOLOv10 để thực hiện đồ án.

3.2.1. So sánh các phiên bản YOLO trên dữ liệu huấn luyện

Để lựa chọn mô hình phù hợp cho bài toán phát hiện khuyết điểm trên PCB trong điều kiện tài nguyên hạn chế của Jetson Nano, tôi đã tiến hành đánh giá ba phiên bản YOLO phổ biến: YOLOv5s, YOLOv8s và YOLOv10s (bao gồm cả biến thể nano của YOLOv10). Việc so sánh được thực hiện dựa trên bộ chỉ số định lượng (số lượng tham số, độ phức tạp tính toán, độ chính xác, tốc độ suy luận) và kết quả thực nghiệm trên ảnh thực tế.

Bảng dưới đây kết hợp các đặc tính cấu tạo của mô hình và kết quả đo lường độ chính xác (mAP) trên tập dữ liệu huấn luyện.

Bảng 3.1. Đặc tính cấu tạo của mô hình

Mô hình	Số Layer	Tham số (triệu)	FLOPs
Yolov5su	85	9.1	23.8
Yolov8s	73	11.0	28.4
Yolov10s	105	7.2	21.4
Yolov10n	102	2.2	6.5

Bảng 3.2. Kết quả đo lường độ chính xác

Mô hình	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Yolov5su	0.9667	0.9224	0.9517	0.5223
Yolov8s	0.9533	0.9353	0.9625	0.5132
Yolov10s	0.9391	0.9385	0.9650	0.4975
Yolov10n	0.8897	0.8387	0.9104	0.5043

Ý nghĩa các thông số độ chính xác:

- **Precision:** đo độ “chính xác” của dự đoán trong tất cả các bbox mà model dự đoán là đúng, có bao nhiêu cái thực sự đúng (giảm false positive).
- **Recall:** đo khả năng “bắt đủ” trong tất cả các object thật có trong ảnh, model phát hiện được bao nhiêu (giảm false negative).
- **mAP@0.5:** mean Average Precision khi ngưỡng IoU = 0.5 từ đó chỉ cần bbox chồng lấp $\geq 50\%$ với ground truth là được tính đúng.
- **mAP@0.5:0.95:** trung bình mAP trên nhiều ngưỡng IoU từ 0.5 đến 0.95 (step 0.05) từ đánh giá khắt khe hơn vì yêu cầu đánh dấu (bounding box) phải khớp rất sát, phản ánh chất lượng định vị tổng thể tốt hơn.

Về cấu trúc và độ phức tạp, YOLOv10s nổi bật với số lượng tham số chỉ 7,2 triệu và thấp hơn đáng kể so với 9,1 triệu của YOLOv5s và 11 triệu của YOLOv8s. Điều này đi kèm với giá trị FLOPs (21,4 tỷ) cũng thấp nhất trong ba phiên bản small, cho thấy tiềm năng về mặt hiệu năng tính toán trên nền tảng nhúng. Trong khi đó, YOLOv8s tuy có số lớp ít hơn (73 lớp so với 105 lớp của YOLOv10s) nhưng lại có tham số và FLOPs cao nhất, dự báo sẽ nặng hơn khi triển khai.

Xét về độ chính xác trên tập dữ liệu PCB, các chỉ số cho thấy sự phân hóa rõ rệt. YOLOv10s đạt mAP@0.5 cao nhất (0,9650), vượt nhẹ so với YOLOv8s (0,9625) và YOLOv5s (0,9516). Tuy nhiên, khi xét mAP@0.5:0.95 chỉ số khắt khe hơn với nhiều mức IoU YOLOv5s lại dẫn đầu (0,5223), tiếp theo là YOLOv8s (0,5132) và YOLOv10s (0,4975). Điều này cho thấy YOLOv10s có khả năng định vị bounding box tốt ở ngưỡng IoU trung bình nhưng lại kém ổn định ở các ngưỡng rất cao. Về recall, YOLOv10s cũng nhỉnh hơn (0,9385) so với YOLOv8s (0,9353) và YOLOv5s (0,9224), trong khi precision của YOLOv10s (0,9391) thấp hơn một chút so với hai mô hình còn lại.

Kết quả kiểm thử trên một ảnh thực tế càng củng cố thêm ưu điểm của YOLOv10s. Trong khi YOLOv5s và YOLOv8s không phát hiện bất kỳ đối tượng

lỗi nào, YOLOv10s đã nhận diện đúng lỗi. Phiên bản YOLOv10n dù phát hiện được 2 lỗi nhưng lại có một số lỗi nhận diện sai nên thể hiện độ tin cậy kém hơn. Như vậy, với cùng một điều kiện đầu vào, YOLOv10s tỏ ra nhạy bén hơn trong việc phát hiện các khuyết điểm có kích thước nhỏ hoặc độ tương phản thấp.

Tốc độ được đo lường trên GPU Tesla T4 (server) và bo mạch nhúng Jetson Nano.

Bảng 3.3. Tốc độ của các mô hình

Mô hình	Tổng thời gian nhận diện 1 ảnh (Tesla T4)	Tổng thời gian nhận diện 1 ảnh (Jetson Nano)
Yolov5su	7.8 ms	380.8 ms
Yolov8s	6.4 ms	442.6 ms
Yolov10s	7.4 ms	377.0 ms
Yolov10n	5.2 ms	197.4 ms

Về tốc độ xử lý, đây là yếu tố sống còn khi triển khai trên Jetson Nano. Trên GPU Tesla T4 (môi trường phát triển), cả bốn mô hình đều có thời gian suy luận dưới 10 ms, trong đó YOLOv10n nhanh nhất (4,8 ms) nhờ số tham số rất thấp. Tuy nhiên, khi chuyển sang Jetson Nano, sự khác biệt trở nên rõ nét hơn. YOLOv10s có tổng thời gian xử lý 377 ms, tương đương YOLOv5s (380,8 ms) và nhanh hơn YOLOv8s (442,6 ms). Điều đặc biệt quan trọng là YOLOv10s có thời gian hậu xử lý (post-processing) chỉ 4,8 ms, thấp hơn nhiều so với 8,2 ms của YOLOv5su và 11,2 ms của YOLOv8s. Lợi thế này đến từ việc YOLOv10 loại bỏ cơ chế Non-Maximum Suppression (NMS) truyền thống, thay bằng phương pháp lấy mẫu đầu ra không cần NMS, giúp giảm đáng kể độ trễ sau suy luận và một điểm nghẽn thường gặp trên các thiết bị nhúng.

Ngoài ra, tôi cũng thử nghiệm giảm kích thước ảnh đầu vào để cải thiện FPS trên Jetson Nano. Với YOLOv10s, khi giảm từ 640 xuống 320 pixel, tổng thời gian xử lý giảm từ 377 ms xuống 130,5 ms, tương đương tốc độ tăng từ khoảng 2–

3 FPS lên 8–10 FPS. Tuy nhiên, độ chính xác suy giảm rõ rệt: mô hình dễ phát hiện nhầm lẫn và bỏ sót nhiều hơn. Khi giảm xuống 160 pixel, mô hình gần như không còn khả năng nhận diện. Do đó, việc cân bằng giữa kích thước ảnh và độ chính xác là một bài toán thiết kế quan trọng, và trong phạm vi nghiên cứu này, tôi quyết định giữ nguyên kích thước đầu vào 640×640 để đảm bảo chất lượng phát hiện, đồng thời tận dụng các tối ưu hóa khác từ mô hình và phần cứng.

Bảng 3.4. Tốc độ mô hình khi giảm kích thước đầu vào

Kích thước ảnh	Tổng thời gian(ms)	Tốc độ (FPS)
640	377	~2-3
320	130.5	~8-10
160	58.5	~15-20

3.2.2. Lý do chọn Yolov10s

Trên cơ sở kết quả so sánh giữa các phiên bản YOLOv5s, YOLOv8s, YOLOv10s và YOLOv10n, việc lựa chọn mô hình YOLOv10s cho hệ thống kiểm tra khuyết điểm PCB trên Jetson Nano được dựa trên ba trụ cột chính: độ chính xác thực tế, hiệu năng xử lý trên nền tảng nhúng và khả năng tối ưu hóa về mặt kiến trúc.

Trước hết, về độ chính xác, mặc dù YOLOv10s có chỉ số $mAP@0.5:0.95$ thấp hơn đôi chút so với YOLOv5s và YOLOv8s, nhưng trong bối cảnh bài toán phát hiện khuyết điểm PCB, độ nhạy (recall) và độ chính xác ở ngưỡng IoU trung bình ($mAP@0.5$) mới là các chỉ số phản ánh sát thực tế vận hành. YOLOv10s đạt recall cao nhất (0,9385) và $mAP@0.5$ cao nhất (0,9650), cho thấy khả năng phát hiện đúng các lỗi tồn tại trên PCB với tỷ lệ sót thấp. Kết quả kiểm thử trên ảnh thực tế đã minh chứng rõ điều này: chỉ có YOLOv10s phát hiện chính xác lỗi open c, trong khi YOLOv5s và YOLOv8s không phát hiện được bất kỳ lỗi nào. Đối với ứng dụng kiểm tra chất lượng sản phẩm, việc bỏ sót lỗi gây hậu quả nghiêm trọng hơn nhiều so với cảnh báo sai do đó ưu tiên recall cao là hoàn toàn phù hợp.

Thứ hai, về hiệu năng trên Jetson Nano, YOLOv10s thể hiện sự cân bằng tối ưu giữa thời gian suy luận và độ chính xác. Với tổng thời gian xử lý 377 ms trên ảnh đầu vào 640×640, mô hình này nhanh hơn đáng kể so với YOLOv8s (442,6 ms) và tương đương với YOLOv5s (380,8 ms). Tuy nhiên, điểm khác biệt then chốt nằm ở thời gian hậu xử lý: YOLOv10s chỉ mất 4,8 ms trong khi YOLOv5s và YOLOv8s lần lượt mất 8,2 ms và 11,2 ms. Khoản tiết kiệm thời gian này đến từ việc YOLOv10 loại bỏ hoàn toàn bước Non-Maximum Suppression (NMS) một phép toán vốn gây nghẽn trên các thiết bị nhúng có tài nguyên tính toán hạn chế. Nhờ đó, hệ thống có thể duy trì tốc độ xử lý ổn định ở mức 2–3 FPS, đáp ứng yêu cầu vận hành của dây chuyền sản xuất thực tế.

Thứ ba, xét đến khả năng mở rộng và tối ưu hóa về mặt kiến trúc, YOLOv10s có số lượng tham số chỉ 7,2 triệu thấp nhất trong nhóm các phiên bản small kéo theo dung lượng bộ nhớ và băng thông truyền dữ liệu giảm đáng kể. Điều này đặc biệt quan trọng trên Jetson Nano vốn có bộ nhớ RAM 4 GB và bộ nhớ đồ họa dùng chung. Mô hình nhẹ hơn không chỉ giảm tải cho bộ nhớ mà còn tạo điều kiện thuận lợi cho việc triển khai đồng thời các tác vụ khác (ví dụ như giao diện người dùng, ghi nhật ký, giao tiếp ngoại vi) mà không gây xung đột tài nguyên. Ngoài ra, việc lựa chọn phiên bản “s” (small) thay vì “n” (nano) là một quyết định có chủ đích. Mặc dù YOLOv10n có thời gian xử lý nhanh hơn (197,4 ms) và số tham số ít hơn, nhưng kết quả kiểm thử thực tế cho thấy độ chính xác của nó kém hơn rõ rệt: precision chỉ đạt 0,8897, recall 0,8387 và mAP@0.5 0,9104. Trên ảnh kiểm thử, YOLOv10n phát hiện sai lỗi, điều này không thể chấp nhận trong bối cảnh kiểm tra chất lượng nghiêm ngặt. Như vậy, YOLOv10n đã đánh đổi quá nhiều độ chính xác để lấy tốc độ, không đáp ứng yêu cầu về độ tin cậy. Ngược lại, YOLOv10s giữ được độ chính xác cao ở mức cần thiết trong khi vẫn đảm bảo hiệu năng vận hành khả thi trên phần cứng.

Tóm lại, YOLOv10s được lựa chọn nhờ sự kết hợp hài hòa giữa khả năng phát hiện lỗi thực tế vượt trội, thời gian hậu xử lý tối ưu nhờ kiến trúc không NMS, kích thước mô hình nhỏ gọn phù hợp với tài nguyên của Jetson Nano, và mức độ

tin cậy cao hơn hẳn so với phiên bản nano. Đây là nền tảng vững chắc để xây dựng hệ thống kiểm tra PCB tự động vừa chính xác vừa ổn định trong điều kiện vận hành liên tục.

3.2.3. Quy trình huấn luyện mô hình

3.2.3.1. Chuẩn bị dữ liệu

Trước khi tiến hành huấn luyện mô hình, tập dữ liệu cần được chuẩn hóa nhằm đảm bảo tính nhất quán và khả năng học hiệu quả của thuật toán. Trong đề tài này, dữ liệu được thu thập từ nguồn công khai trên Kaggle, bao gồm nhiều ảnh PCB chứa các loại lỗi phổ biến như short circuit, open circuit, missing hole, spur, mouse bite và spurious copper.

Tập dữ liệu ban đầu được kiểm tra và loại bỏ các ảnh bị lỗi, ảnh mờ hoặc nhãn không chính xác. Sau đó, toàn bộ dữ liệu được chuyển đổi sang định dạng YOLO gồm hai thành phần: ảnh đầu vào (.jpg) và tệp nhãn (.txt). Trong đó, mỗi tệp nhãn chứa thông tin về lớp đối tượng và tọa độ bounding box được chuẩn hóa theo kích thước ảnh.

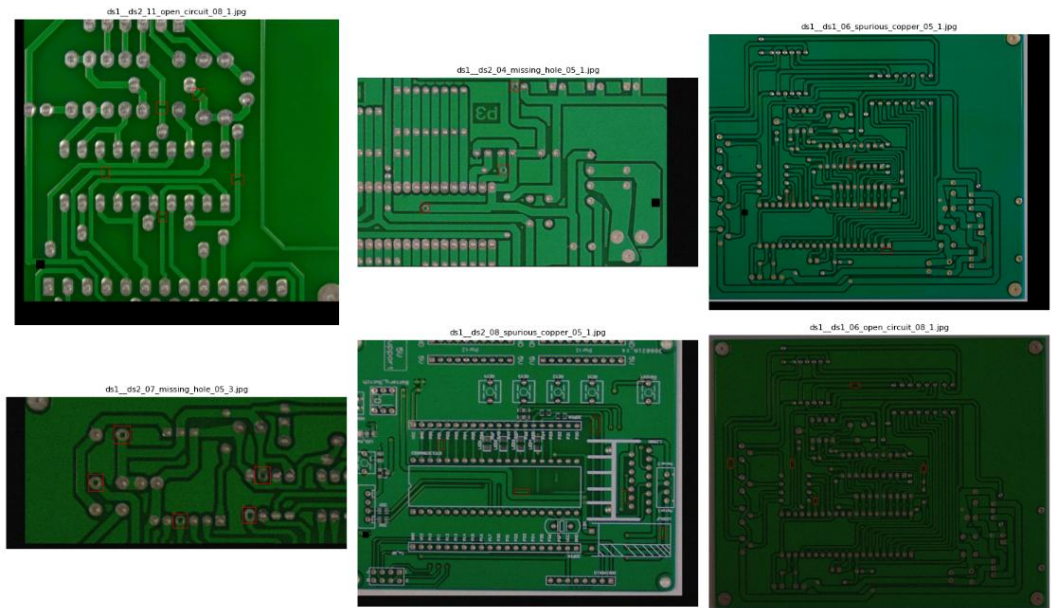
Nhằm đảm bảo mô hình có khả năng tổng quát hóa tốt, tập dữ liệu được chia thành ba phần:

Training set (80%): dùng để cập nhật trọng số mô hình

Validation set (10%): dùng để đánh giá trong quá trình huấn luyện

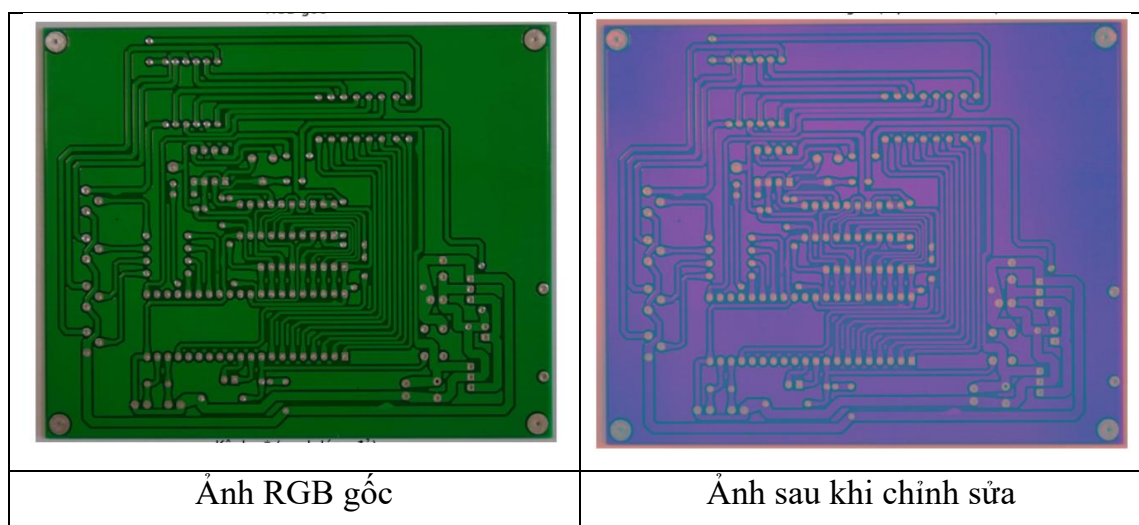
Test set (10%): dùng để kiểm tra cuối cùng

Việc phân chia dữ liệu giúp hạn chế hiện tượng overfitting và đảm bảo kết quả đánh giá khách quan. Một số mẫu ảnh như sau:



Hình 3.2. Một số mẫu PCB trong tập dữ liệu

Tập dữ liệu được chia 80% cho tập huấn luyện, 10% cho tập xác thực(validation) và còn lại cho tập kiểm tra. Ngoài ra, tôi còn chuyển không gian màu từ RGB sang một không gian màu lớn hơn là CIELab. Lý do cho việc áp dụng LAB cho bài toán nhận diện lỗi PCB để khắc phục các điều kiện ánh sáng không đồng đều trên bo mạch, đồng thời làm nổi bật các các điểm bất thường. Hơn thế nữa, tôi tách thông tin độ sáng và làm rõ nét bằng CLAHE sau đó gộp lại các kênh a^* và b^* . Ví dụ một kết quả như sau:



Hình 3.3. Ảnh trong tập dữ liệu sau khi qua xử lý

Ảnh sau khi chỉnh sửa trông sai màu với mắt người, nhưng ảnh sẽ tốt cho mô hình vì nó giúp mô hình tập trung vào cấu trúc, hình khối và độ tương phản của vật thể mà không bị đánh lừa bởi điều kiện ánh sáng cho môi trường khi môi trường quá sáng hoặc quá chói gây ảnh hưởng tới chất lượng ảnh đầu vào.

3.2.3.2. Huấn luyện mô hình

Do số lượng ảnh lỗi PCB thực tế còn hạn chế, kỹ thuật tăng cường dữ liệu được áp dụng nhằm tạo thêm nhiều biến thể khác nhau từ dữ liệu gốc, giúp mô hình học được tính bất biến của đối tượng.

Các phép biến đổi được sử dụng gồm:

- Xoay ảnh ngẫu nhiên nhiều góc
- Lật ngang, lật dọc
- Thay đổi độ sáng và độ tương phản
- Phóng to, thu nhỏ Dịch chuyển vị trí đối tượng
- Mosaic augmentation (ghép nhiều ảnh)

Các phương pháp này giúp mô hình tăng khả năng thích nghi với điều kiện chụp thực tế như thay đổi ánh sáng, góc đặt camera hoặc sai lệch vị trí của PCB.

Để tăng hiệu quả huấn luyện, tôi sử dụng mô hình YOLOv10 đã được huấn luyện trước trên tập dữ liệu COCO (pre-trained model), cho phép kế thừa các đặc trưng hình ảnh mà mô hình đã học trước đó. Nhờ đó, quá trình hội tụ diễn ra nhanh hơn và hiệu suất nhận diện được cải thiện đáng kể. Chi tiết các tham số được đưa vào mô hình được trình bày trong hình bên dưới.

```

model = YOLO('yolov10s.pt')

results = model.train(
    data=LAB_DATA_YAML,
    epochs=50,
    imgsz=640,
    batch=16,
    device=[0,1],
    # --- Tăng cường hình học (giữ nguyên từ bản gốc) ---
    degrees=15.0,      # mặc định 0.0 - xoay theo độ, ±15°
    translate=0.1,     # mặc định 0.1 - giữ nguyên
    scale=0.9,         # mặc định 0.5 - dải tỷ lệ rộng hơn
    shear=5.0,         # mặc định 0.0 - cắt xén ±5°, bắt chước ngoài trục
    perspective=0.0005, # mặc định 0.0 - biến dạng phối cảnh nhỏ nhưng rõ ràng
    flipud=0.5,        # mặc định 0.0 - lật dọc OK cho PCB từ trên xuống
    fliplr=0.5,
    mosaic=1.0,
    mixup=0.15,        # mặc định 0.0 - pha trộn vừa phải
    copy_paste=0.3,    # mặc định 0.0 - dán các lỗi qua các hình ảnh

```

Hình 3.4. Thông số huấn luyện mô hình

3.2.3.3. Kết quả huấn luyện

Sau thời gian huấn luyện, mô hình cho kết quả tốt. Các đường line 0 đến 5 lần lượt với các lỗi sau missing hole, mouse bite, open circuit, short, spur, spur copper. Đầu tiên, phân tích kết quả của ảnh Confusion Matrix Normalized.



Hình 3.5. Confusion matrix normalized

Qua ảnh kết quả, tôi nhận xét kết quả như sau:

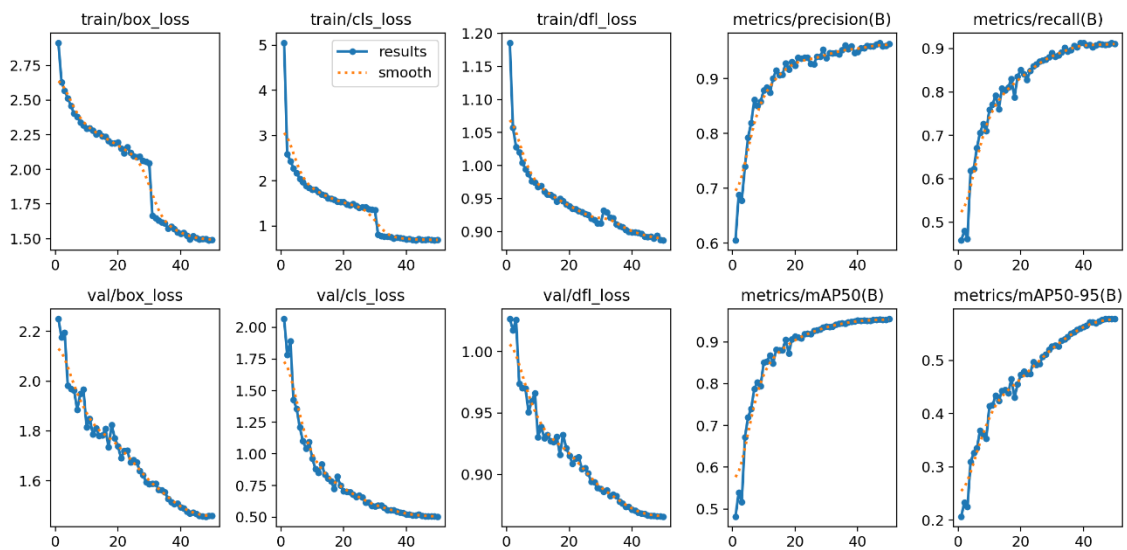
Bảng 3.5. Nhận xét kết quả Confusion Matrix Normalized

Nhóm	Phân loại	Độ chính xác	Đánh giá
0	Missing hole	94%	Rất tốt
1	Mouse bite	87%	Tốt
2	Open circuit	95%	Rất tốt
3	Short	98%	Rất tốt
4	Spur	86%	Tốt
5	Spur copper	96%	Rất tốt

Qua bảng, các lỗi missing hole, open circuit, short và spur copper đạt được kết quả rất tốt cho thấy độ chính xác khi nhận diện các lỗi này cao không bị nhầm lẫn với các lỗi khác. Mặc dù lỗi open circuit nhầm lẫn với lỗi mouse bit nhưng tỉ

lệ rất thấp chỉ 0.01%. Song đó, lỗi mouse bit và spur có lần lượt tỉ lệ dự đoán sai và bỏ sót lần lượt là 12% và 13%. Cho thấy 2 lỗi này là thách thức lớn vì 2 lỗi này khá khó nhận biết vì lỗi nó không rõ ràng như các lỗi ở trên.

Các chỉ số mô hình sau khi train cũng rất ấn tượng với Precision đạt 0.9636, Recall đạt 0.91075, mAP50 đạt 0.95513 và mAP50:95 là 0.57817. Hình ảnh ở dưới tổng hợp các kết quả như sau:



Hình 3.6. Kết quả tổng quát của mô hình

3.3. KIẾN TRÚC TỔNG QUAN CỦA HỆ THỐNG

Hệ thống được thiết kế theo kiến trúc phân lớp, hướng đến tính mô-đun hóa và khả năng mở rộng, phù hợp với nền tảng nhúng NVIDIA Jetson Nano. Toàn bộ hệ thống bao gồm ba khối chính: khối thu thập dữ liệu, khối xử lý trung tâm, và khối hiển thị và tương tác. Sự kết nối giữa các khối được thể hiện qua một luồng dữ liệu khép kín, đảm bảo độ trễ thấp và hoạt động liên tục.

Khối thu thập dữ liệu đảm nhận việc tiếp nhận hình ảnh từ camera (CSI hoặc USB) dưới dạng luồng video thời gian thực. Tín hiệu hình ảnh sau khi được số hóa sẽ được chuyển trực tiếp đến khối xử lý trung tâm thông qua các thư viện

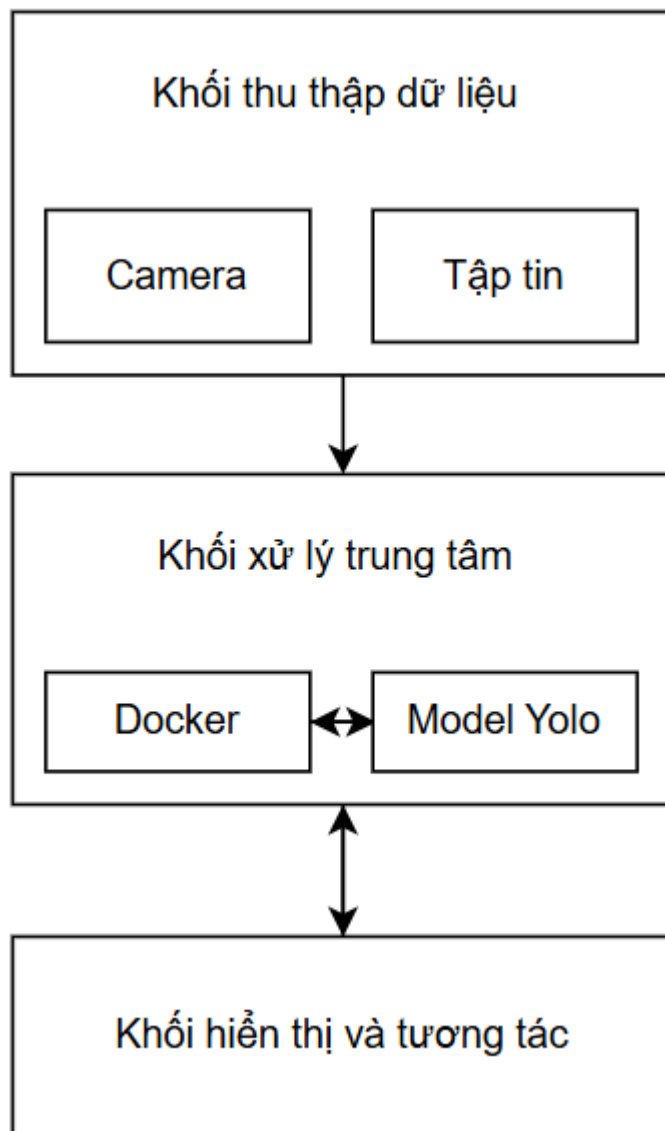
hỗ trợ phần cứng như GStreamer hoặc V4L2, tận dụng khả năng xử lý đa phương tiện tích hợp trên Jetson Nano.

Khối xử lý trung tâm là trái tim của hệ thống, nơi mô hình nhận diện khuyết điểm YOLOv10s được triển khai. Khối này hoạt động hoàn toàn trong môi trường container Docker, với runtime GPU được kích hoạt qua NVIDIA Container Toolkit. Nhờ đó, mô hình có thể truy cập trực tiếp vào CUDA cores, tối ưu hóa tốc độ inference. Luồng xử lý diễn ra tuần tự: ảnh đầu vào được chuẩn hóa (resize, chuẩn hóa màu), sau đó đưa qua mô hình đã được tối ưu để phát hiện và phân loại các lỗi như short circuit, open circuit, missing hole... Kết quả trả về bao gồm tọa độ bounding box, nhãn lỗi và độ tin cậy.

Khối hiển thị và tương tác được xây dựng dưới dạng Python tkinter để tối ưu hóa hiệu suất. Khối này nhận kết quả từ khối xử lý, hiển thị trực tiếp lên màn hình kèm theo các cảnh báo trực quan. Đồng thời, khối cũng cung cấp các thành phần điều khiển như nút khởi động/dừng giám sát, xem lịch sử, xuất báo cáo. Nhờ tách biệt khỏi logic xử lý, khối hiển thị có thể dễ dàng thay thế hoặc nâng cấp mà không ảnh hưởng đến lõi nhận diện.

Mối liên hệ giữa phần cứng Jetson Nano và môi trường Docker thể hiện rõ trong cách tổ chức tài nguyên. Jetson Nano đóng vai trò là nền tảng vật lý cung cấp GPU Maxwell, bộ nhớ chia sẻ và các giao tiếp ngoại vi. Docker đóng vai trò là lớp ảo hóa, bao bọc toàn bộ môi trường phụ thuộc (CUDA, cuDNN, thư viện Python) thành một image nhất quán. Khi hệ thống khởi động, container được chạy với các quyền truy cập thiết bị để đọc camera và sử dụng GPU. Cách tiếp cận này đảm bảo tính tái lập cao, giảm thiểu xung đột thư viện, đồng thời cho phép dễ dàng triển khai trên nhiều thiết bị Jetson khác nhau mà không cần cài đặt lại.

Sơ đồ khối tổng quát của hệ thống được minh họa qua luồng sau:



Hình 3.7. Sơ đồ khối tổng quát

3.4. XÂY DỰNG GIAO DIỆN NGƯỜI DÙNG VÀ TRIỂN KHAI TRÊN DOCKER

Giao diện người dùng đóng vai trò cầu nối giữa hệ thống nhận diện khuyết điểm và người vận hành. Trong bối cảnh kiểm tra PCB, giao diện cần đảm bảo tính trực quan, phản hồi nhanh và hỗ trợ tác vụ giám sát liên tục. Dựa trên yêu cầu về tính đóng gói và khả năng triển khai trên Jetson Nano, giao diện được xây dựng với các lựa chọn công nghệ và cấu trúc phù hợp.

3.4.1. Lựa chọn công nghệ

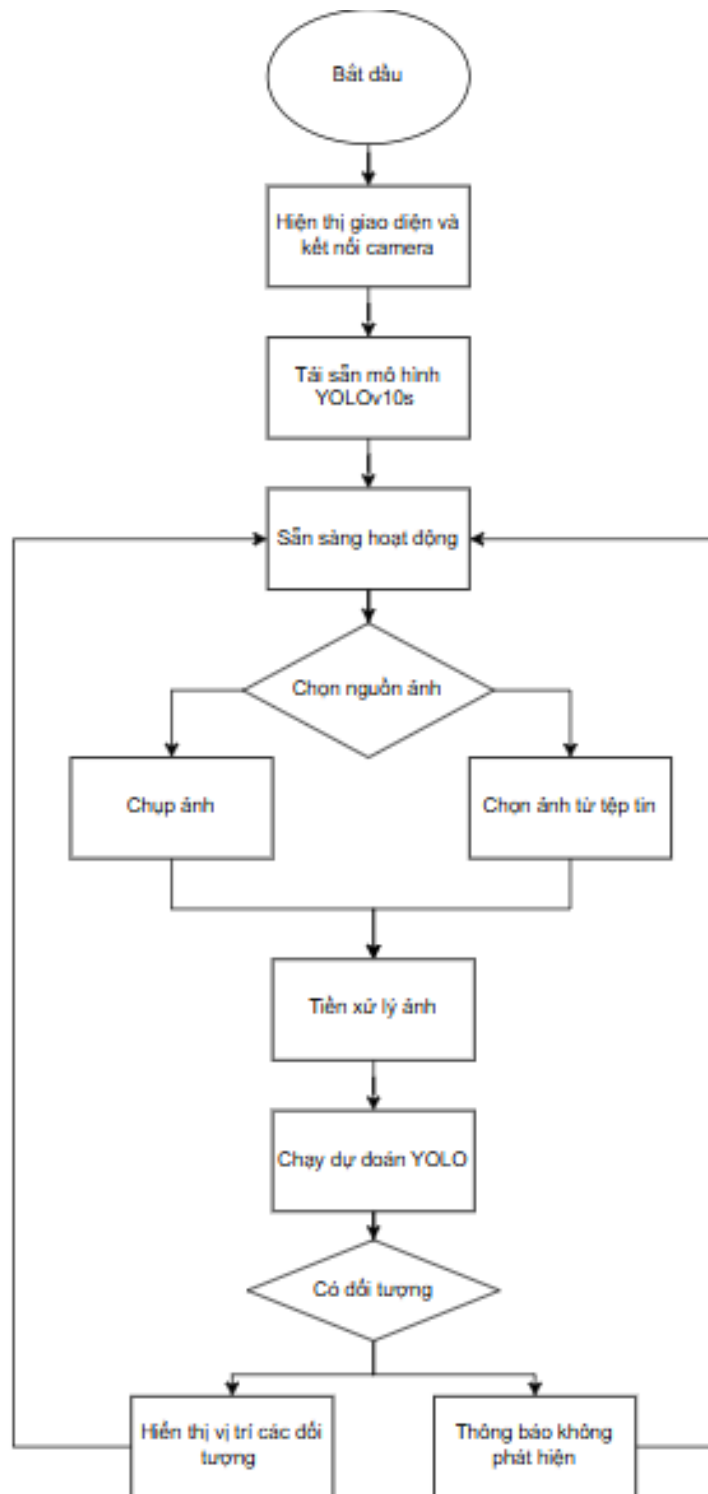
Giao diện được phát triển bằng Tkinter, thư viện đồ họa chuẩn của Python, kết hợp với Pillow (PIL) để xử lý và hiển thị hình ảnh, và OpenCV để truy xuất khung hình từ camera cũng như vẽ các bounding box lên ảnh kết quả. Sự lựa chọn này dựa trên các lý do sau:

- Tính nhẹ và ổn định trên nền tảng nhúng: Tkinter là thư viện gốc của Python, không yêu cầu cài đặt thêm các framework nặng như web server hay trình duyệt. Điều này giảm tải bộ nhớ và CPU, phù hợp với tài nguyên hạn chế của Jetson Nano.
- Tích hợp dễ dàng với OpenCV và YOLO: Việc chuyển đổi giữa định dạng ảnh của OpenCV (numpy array) và Tkinter (PhotoImage) được thực hiện qua Pillow một cách mượt mà, không phát sinh độ trễ đáng kể.
- Không phụ thuộc vào mạng: Khác với các giải pháp dựa trên web (Flask, Streamlit), ứng dụng Tkinter chạy cục bộ, không yêu cầu trình duyệt hay kết nối mạng nội bộ, giảm thiểu các điểm có thể gây lỗi trong môi trường sản xuất.

Ngoài ra, giao diện được thiết kế với phong cách tối giản, sử dụng ttk (themed Tkinter) để đồng nhất màu sắc và kiểu dáng, đồng thời hỗ trợ tùy chỉnh giao diện phù hợp với thương hiệu hoặc sở thích người dùng.

3.4.2. Xây dựng giao diện người dùng

Giao diện được tổ chức theo một luồng tương tác xoay quanh việc hiển thị trực tiếp từ camera và các thao tác kiểm tra theo yêu cầu. Sơ đồ dưới đây mô tả các trạng thái và hành vi chính:



Hình 3.8. Sơ đồ hoạt động của chương trình

1. Trạng thái mặc định Xem trực tiếp từ camera: Khi ứng dụng khởi động, camera được kích hoạt và luồng hình ảnh liên tục hiển thị trong khung

chính và bắt đầu nạp mô hình Yolov10s. Người vận hành có thể quan sát các sản phẩm PCB di chuyển trên dây chuyền. Ở trạng thái này, nút “Camera” (Back) bị vô hiệu hóa vì không có kết quả nhận diện nào đang được hiển thị.

2. Kích hoạt nhận diện: Người dùng có hai cách để đưa ảnh vào hệ thống: Chụp & Predict: Lấy khung hình hiện tại từ camera và gửi đến mô hình YOLO. Thao tác này thường được sử dụng khi dây chuyền tạm dừng để kiểm tra mẫu. Chọn ảnh: Mở hộp thoại chọn file ảnh từ bộ nhớ, cho phép kiểm tra lại các sản phẩm đã lưu hoặc ảnh chụp từ nguồn khác. Sau khi nhận lệnh, giao diện chuyển sang trạng thái “busy”: các nút bị vô hiệu hóa, thanh trạng thái hiển thị thông báo “Đang predict...” để tránh thao tác đồng thời.
3. Hiển thị kết quả nhận diện: Khi mô hình trả về kết quả, ảnh gốc được vẽ các bounding box và nhãn tương ứng với từng khuyết điểm. Ảnh đã được ghi chú hiển thị trong khung chính, thay thế luồng camera. Đồng thời, bảng danh sách (Treeview) bên dưới được cập nhật chi tiết từng đối tượng phát hiện, bao gồm nhãn lỗi, độ tin cậy và tọa độ bounding box. Nút “Camera” trở nên khả dụng để người dùng quay lại chế độ xem trực tiếp.
4. Tương tác với danh sách đối tượng: Khi người dùng nhấp vào một dòng trong bảng, bounding box tương ứng sẽ được tô sáng bằng màu xanh lục đậm hơn, giúp xác định vị trí chính xác của khuyết điểm trên ảnh. Tính năng này hỗ trợ người vận hành kiểm tra lại nhanh chóng các lỗi nghi ngờ.
5. Quay lại chế độ camera: Nhấn nút “Camera” sẽ đưa giao diện về trạng thái xem trực tiếp, xóa kết quả cũ và cho phép thực hiện kiểm tra mới.

Luồng tương tác được thiết kế theo hướng tối giản, giảm thiểu số thao tác cần thiết để thực hiện một lần kiểm tra, đáp ứng yêu cầu vận hành nhanh trong môi trường sản xuất.

3.4.3. Thiết kế các thành phần UI

Giao diện chính của ứng dụng được chia làm bốn khu vực rõ ràng, bố trí theo chiều dọc:

Tiêu đề: Nằm ở đầu cửa sổ, hiển thị tên ứng dụng “YOLO Object Detection” với font chữ đậm, kích thước lớn, tạo điểm nhấn nhận diện.

Khung hiển thị hình ảnh: Chiếm phần lớn diện tích, có nền màu xám đậm để làm nổi bật ảnh đầu vào. Kích thước ảnh được điều chỉnh đồng bộ thành 800×450 pixel bằng phương pháp LANCZOS, đảm bảo tỷ lệ và chất lượng hiển thị. Khu vực này cập nhật động giữa luồng camera và ảnh kết quả.

Thanh công cụ: Gồm ba nút bấm chính:

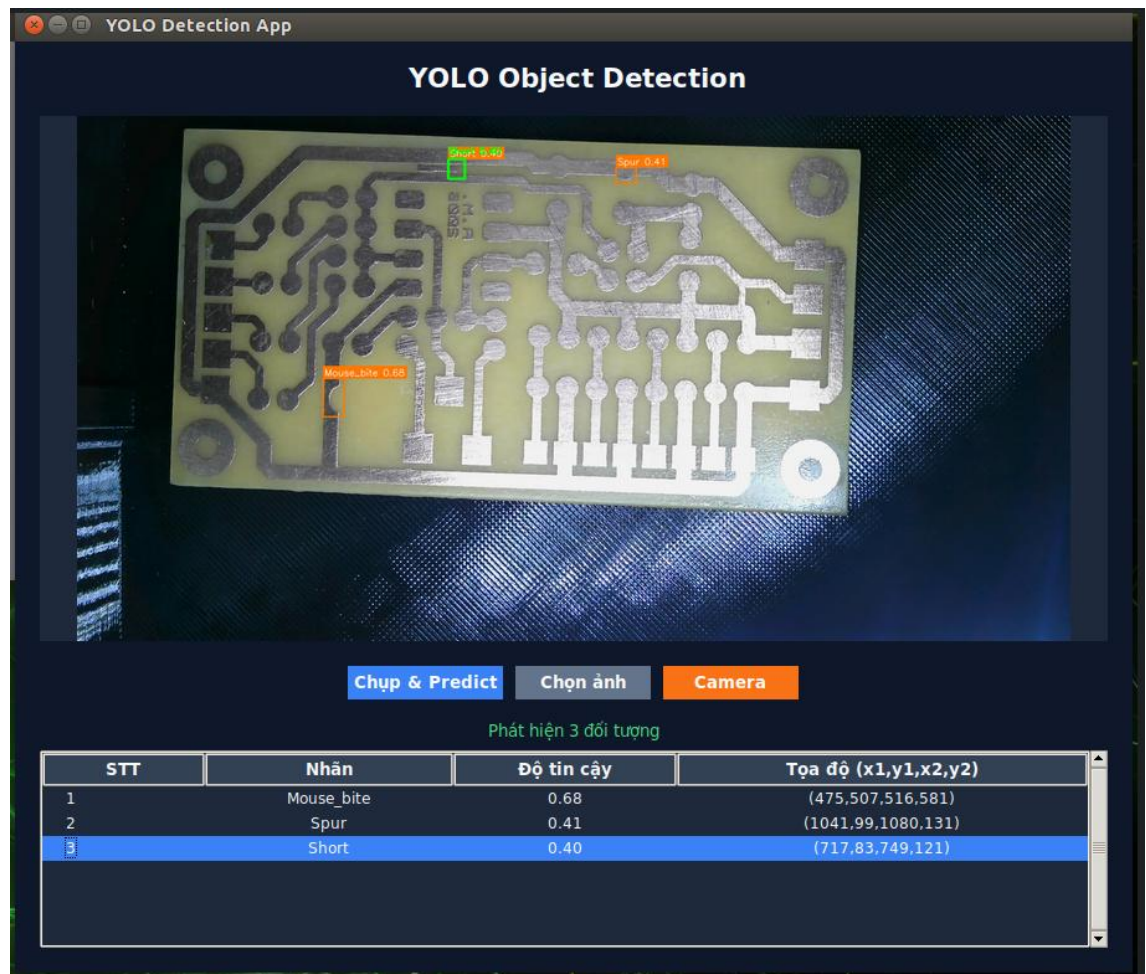
- Chụp & Predict : Dùng để lấy ảnh từ camera hiện tại và thực hiện nhận diện.
- Chọn ảnh: Mở hộp thoại chọn file ảnh từ hệ thống.
- Camera : Chỉ khả dụng khi đang xem kết quả, cho phép quay lại chế độ xem trực tiếp.

Các nút có trạng thái enabled/disabled rõ ràng để ngăn người dùng thực hiện hành động không hợp lệ. Khi đang xử lý, tất cả nút đều bị vô hiệu hóa, chỉ trừ nút “Camera” sẽ được kích hoạt lại sau khi có kết quả.

Thanh trạng thái: Nằm ngay dưới thanh công cụ, hiển thị thông báo về trạng thái hệ thống (sẵn sàng, đang xử lý, số lượng đối tượng phát hiện, lỗi). Màu chữ thay đổi theo mức độ quan trọng: xanh lục cho thông báo bình thường, vàng cho đang xử lý, đỏ cho lỗi.

Bảng danh sách đối tượng (Treeview): Đặt ở cuối cửa sổ, hiển thị chi tiết từng khuyết điểm với các cột: STT, Nhân, Độ tin cậy, Tọa độ (x1,y1,x2,y2). Bảng có thanh cuộn dọc để hỗ trợ khi số lượng đối tượng lớn. Khi người dùng click chọn

một dòng, sự kiện được xử lý để làm nổi bật bounding box tương ứng trên ảnh, giúp dễ dàng xác định vị trí lỗi.



Hình 3.9. Giao diện ứng dụng

Nhờ thiết kế tập trung vào trải nghiệm người dùng trong môi trường sản xuất, giao diện vừa đáp ứng yêu cầu giám sát thời gian thực, vừa hỗ trợ kiểm tra chi tiết khi phát hiện lỗi, đồng thời dễ dàng đóng gói và triển khai trên nền tảng Jetson Nano.

3.4.4. Triển khai ứng dụng

Để đảm bảo ứng dụng có thể vận hành ổn định trên hệ thống Linux của Jetson Nano, đồng thời mang lại trải nghiệm người dùng thuận tiện và không yêu cầu can thiệp sâu vào dòng lệnh mỗi khi khởi động, tôi đã thiết kế một cơ chế triển

khai dựa trên Docker kết hợp với script khởi chạy và lỗi tắt trên màn hình desktop. Cách tiếp cận này vừa đáp ứng yêu cầu về tính đóng gói (dependency isolation) vừa đảm bảo khả năng tương tác đồ họa trực tiếp với môi trường X11 của máy chủ.

Toàn bộ quá trình triển khai được xây dựng xoay quanh một script shell có tên *launch_yolo.sh*, đóng vai trò là điểm khởi đầu duy nhất để khởi động toàn bộ hệ thống. Script này thực hiện ba nhiệm vụ chính. Trước tiên, nó cấp quyền cho các container Docker có thể kết nối đến máy chủ X11 thông qua lệnh *xhost +*, điều này rất quan trọng để giao diện Tkinter có thể hiển thị trên màn hình của hệ thống chủ. Tiếp theo, script kiểm tra trạng thái của dịch vụ Docker; nếu chưa được khởi động, nó tự động kích hoạt bằng *systemctl start docker* và chờ một khoảng thời gian ngắn để đảm bảo dịch vụ sẵn sàng. Cuối cùng, script thực thi câu lệnh *docker run* với các tùy chọn thiết yếu: *--runtime=nvidia* để kích hoạt GPU qua NVIDIA Container Runtime, *--ipc=host* để chia sẻ không gian bộ nhớ liên tiến trình, gắn kết các thư mục chứa mô hình (*~/models*), dữ liệu (*~/data*) và mã nguồn (*~/app*) vào bên trong container, truyền biến môi trường *DISPLAY* và mount socket X11, cùng với việc chỉ định thiết bị camera */dev/video0*. Container được khởi chạy dựa trên image *ultralytics/ultralytics:latest-jetson-jetpack4* – một image được tối ưu sẵn cho Jetson Nano với CUDA và các thư viện cần thiết. Lệnh khởi chạy bên trong container là *cd /workspace/app && python3 appv2.py*, tức là chuyển đến thư mục ứng dụng và chạy chương trình Python đã được viết sẵn.

Để người vận hành không cần mở terminal mỗi lần, nhóm đã tạo một tệp *.desktop* trên màn hình với tên *YOLO.desktop*. Tệp này chứa các thông tin cơ bản như tên ứng dụng, biểu tượng, và quan trọng nhất là lệnh thực thi trỏ đến script *launch_yolo.sh* với tham số *Terminal=true* để hiển thị cửa sổ terminal trong quá trình chạy, giúp dễ dàng quan sát nhật ký và xử lý lỗi. Sau khi hệ thống hoạt động ổn định, có thể chuyển sang *Terminal=false* để giao diện khởi chạy hoàn toàn ẩn, chỉ hiện cửa sổ ứng dụng chính.

Một vấn đề phát sinh trong quá trình triển khai là quyền thực thi các lệnh `sudo` khi khởi động Docker. Để tránh việc người dùng phải nhập mật khẩu mỗi lần, nhóm đã cấu hình file `/etc/sudoers` bằng lệnh `visudo`, thêm một dòng cho phép người dùng hiện tại thực thi `docker` và `systemctl start docker` mà không cần xác thực. Biện pháp này đảm bảo tính tự động hóa trong khi vẫn giữ an toàn hệ thống vì chỉ cấp quyền cho hai lệnh cụ thể.

Kết quả của quy trình triển khai này là một hệ thống hoàn chỉnh mà người dùng cuối chỉ cần double-click vào biểu tượng trên màn hình để khởi chạy. Mọi thành phần từ môi trường ảo hóa, thư viện phụ thuộc, tối ưu GPU, đến giao diện người dùng đều được tự động thiết lập mà không cần can thiệp thủ công. Cách tiếp cận này không chỉ phù hợp với tính chất nhúng của Jetson Nano mà còn dễ dàng chuyển giao, nâng cấp và bảo trì trong tương lai.

3.4.5 Hướng dẫn sử dụng hệ thống

Để đảm bảo người vận hành có thể nhanh chóng làm quen và sử dụng hệ thống một cách hiệu quả, quy trình kiểm tra một bo mạch PCB được chuẩn hóa theo các bước sau:

Bước 1 Khởi động hệ thống: Nhấp đúp chuột vào biểu tượng "YOLO App" trên màn hình desktop của Jetson Nano. Hệ thống sẽ tự động khởi động Docker container và tải mô hình YOLOv10s vào bộ nhớ GPU. Quá trình này thường mất khoảng 2 phút. Sau khi sẵn sàng, cửa sổ giao diện chính hiện ra kèm theo luồng hình ảnh trực tiếp từ camera.

Bước 2 Đặt bo mạch PCB vào buồng kiểm tra: Mở cơ cấu cửa trượt ở tầng hạ của mô hình và đặt bo mạch cần kiểm tra vào đúng vị trí trên bệ đỡ bên trong buồng tối. Đóng cửa lại để đảm bảo ánh sáng bên ngoài không lọt vào làm nhiễu ảnh. Trên giao diện, người dùng có thể quan sát hình ảnh bo mạch hiển thị trực tiếp để kiểm tra vị trí đặt bo mạch đã cân đối chưa.

Bước 3 Thực hiện nhận diện: Nhấn nút "Chụp & Predict" để hệ thống lấy khung hình hiện tại từ camera, thực hiện tiền xử lý ảnh (chuyển đổi không gian màu LAB, chuẩn hóa kích thước) và đưa vào mô hình suy luận. Trong khi xử lý, tất cả nút bấm bị vô hiệu hóa và thanh trạng thái hiển thị thông báo "Đang predict...". Người dùng cũng có thể chọn ảnh từ bộ nhớ thay vì chụp trực tiếp bằng cách nhấn nút "Chọn ảnh".

Bước 4 Đọc kết quả: Sau khoảng 400ms, ảnh kết quả hiển thị trên khung chính kèm các bounding box màu khác nhau cho từng loại lỗi. Bảng danh sách bên dưới liệt kê chi tiết từng lỗi phát hiện bao gồm tên lỗi, độ tin cậy và tọa độ vị trí. Người dùng có thể nhấp vào từng dòng trong bảng để làm nổi bật bounding box tương ứng trên ảnh, giúp xác định chính xác vị trí lỗi trên bo mạch.

Bước 5 Xử lý sau nhận diện: Dựa trên kết quả, người vận hành quyết định giữ lại bo mạch để sửa chữa hoặc loại bỏ. Nhấn nút "Camera" để quay về chế độ xem trực tiếp, lấy bo mạch ra và tiến hành kiểm tra bo mạch tiếp theo.

3.5 THIẾT KẾ MÔ HÌNH 3D

3.5.1 Yêu cầu về thiết kế mô hình 3D

Dựa trên mục đích nghiên cứu, đề tài hướng tới xây dựng một hệ thống kiểm tra bán thủ công, phù hợp với quy mô ứng dụng tại các doanh nghiệp vừa và nhỏ. Mục tiêu cốt lõi của hệ thống là phát hiện lỗi trên các bo mạch in (PCB) kích thước 10 x 10 cm thông qua kỹ thuật xử lý và phân tích hình ảnh đầu vào. Để hiện thực hóa mục tiêu này, thiết kế phần cứng và cấu trúc mô hình cần đáp ứng các yêu cầu cơ bản sau:

- Buồng chụp ảnh chuyên dụng (Buồng tối): Thiết kế một buồng kín nhằm cô lập hoàn toàn vùng không gian chụp ảnh với ánh sáng nhiễu từ môi trường bên ngoài, đảm bảo điều kiện chiếu sáng ổn định.

- Hệ thống chiếu sáng chống lóa: Do bề mặt mạch đồng có độ phản xạ ánh sáng cao dễ gây ra hiện tượng lóa (nhiều ảnh) khi chiếu sáng trực tiếp, hệ thống cần tích hợp thêm các tấm tán sáng để phân bổ nguồn sáng đều và mịn lên bề mặt PCB.
- Bố trí Camera tối ưu: Vị trí lắp đặt camera phải được tính toán phân kỳ hợp lý, đảm bảo bao quát toàn bộ diện tích bo mạch cần kiểm tra mà không làm biến dạng hình ảnh thu nhận.
- Khối xử lý trung tâm: Khu vực cơ bản của mô hình được thiết kế làm nơi bố trí kit máy tính nhúng NVIDIA Jetson Nano đóng vai trò trung tâm tiếp nhận dữ liệu hình ảnh từ camera và thực thi mô hình dự đoán lỗi.

3.5.2 Phân tích thiết kế mô hình 3D

Để đáp ứng trọn vẹn các yêu cầu kỹ thuật đã đặt ra ở mục 3.5.1, mô hình vật lý của hệ thống được đề xuất thiết kế theo cấu trúc module dạng đứng, phân chia thành 3 tầng chức năng riêng biệt từ cao xuống thấp. Cách bố trí này giúp tối ưu hóa không gian, đảm bảo tính thẩm mỹ và hỗ trợ hệ thống vận hành ổn định:

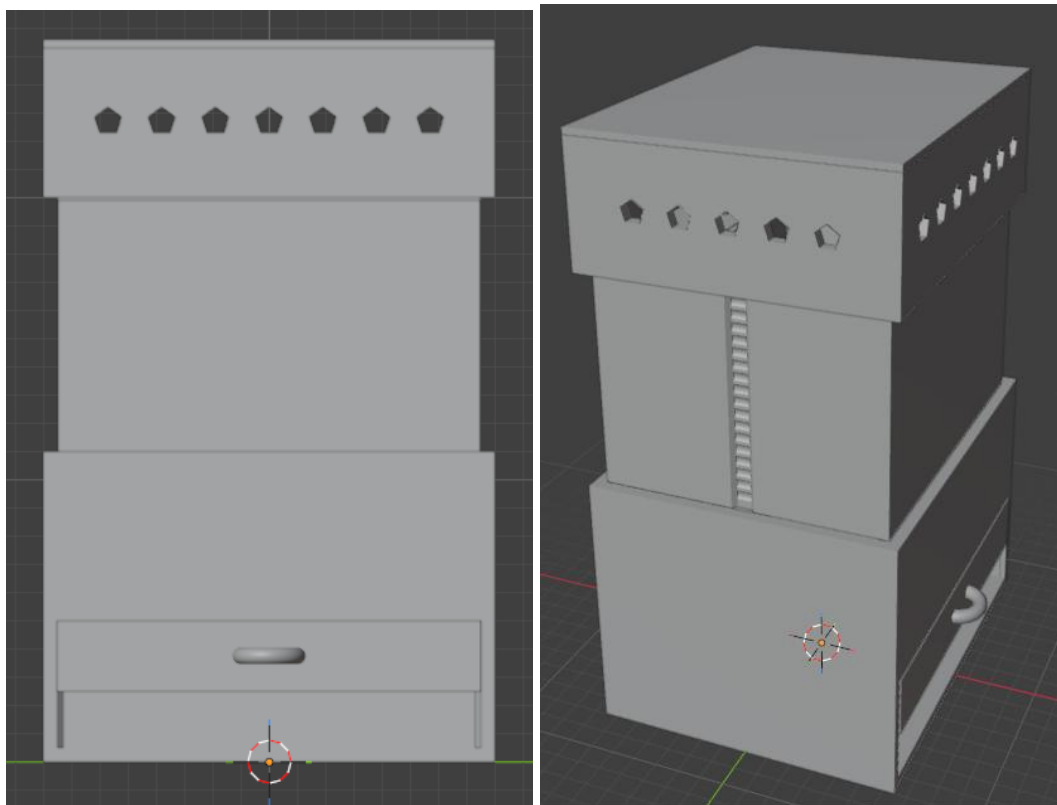
- Tầng thượng (Tầng trung tâm điều khiển): Vị trí cao nhất của mô hình được dành riêng để bố trí kit máy tính nhúng Jetson Nano cùng hệ thống mạch nguồn cấp. Việc đặt trung tâm xử lý ở tầng trên cùng giúp cách ly thiết bị điện tử khỏi vùng thao tác cơ học, tạo điều kiện thuận lợi cho việc tản nhiệt tự nhiên và dễ dàng kết nối các cổng ngoại vi, nút nhấn điều khiển hoặc màn hình hiển thị.
- Tầng trung gian (Tầng camera): Đây là phân khu cố định camera thu hình. Vị trí này được tính toán khoảng cách chuẩn xác so với đáy buồng tối, đảm bảo ống kính camera có góc nhìn bao quát toàn diện, tiêu cự tối ưu để chụp lại trọn vẹn bo mạch 10 x 10 cm với độ nét cao nhất mà không bị méo góc.
- Tầng hạ (Tầng buồng tối và thao tác): Là không gian kín hoàn toàn nhằm cô lập ánh sáng môi trường. Bên trong tầng này tích hợp hệ thống đèn LED và tấm tán sáng chống lóa, cùng với bộ đỡ cố định bo mạch PCB cần kiểm

tra. Đây cũng là nơi người vận hành đưa mạch vào hoặc lấy mạch ra một cách dễ dàng trong quy trình bán thủ công.

Sự liên kết chặt chẽ giữa 3 tầng chức năng tạo nên một chỉnh thể mô hình kiểm tra bo mạch hoàn chỉnh, khép kín từ khâu chiếu sáng, thu nhận hình ảnh cho đến xử lý dữ liệu.

3.5.3 Kết quả thiết kế mô hình 3D

Sau khi thời gian nghiên cứu và đánh giá thì tôi chốt thiết kế mô hình như sau:



Hình 3.10. Hình ảnh tổng quan thiết kế

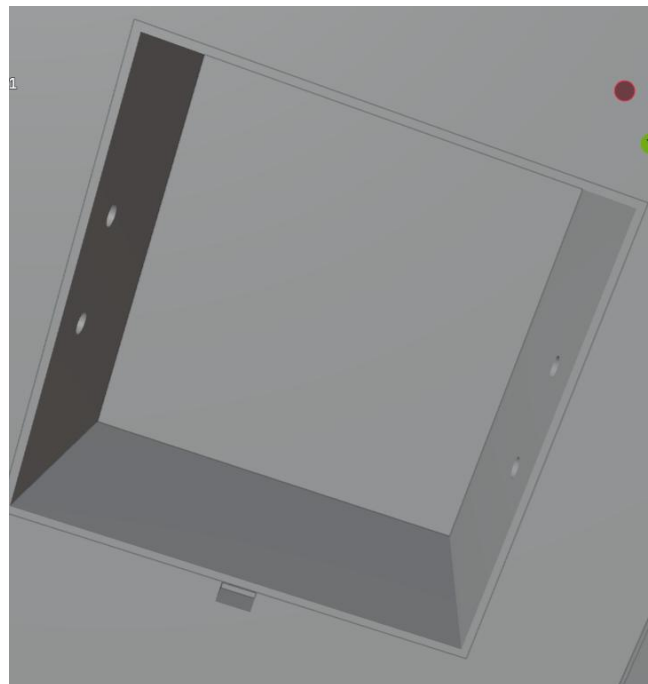
Chi tiết kích thước thiết kế như sau theo trục xyz theo đơn vị milimet lần lượt như sau:

- Tầng thượng: 130,160,53

- Tầng trung gian: 120,150,90
- Tầng hạ:130,160,110
- Tổng thể: 130,160, 260

Thiết kế mô hình được chia làm ba tầng rõ ràng, ở tầng thượng có các lỗ để lưu thông không khí ngoài ra còn có thể làm đầu ra các dây kết nối. Tầng trung gian là tầng chứa camera, đèn và tấm tản sáng được thiết kế tách biệt với buồng tối có thể nâng chỉnh độ cao tùy theo như cầu của người dùng. Tầng hạ được thiết kế có khe có cơ chế đóng mở để đưa mạch in PCB vào và lấy ra.

Vị trí đặt camera nằm dưới tầng thượng có lỗ để đi dây lên bên trên kết nối với trung tâm xử lý. Chiều đặt camera sẽ hướng từ trên xuống, camera sẽ được trang bị một vòng đèn led hỗ trợ ánh sáng và một tấm tản sáng.



Hình 3.11. Vị trí đặt camera

Sau khi dùng mô hình được thiết kế đi in 3D, tôi thu được kết quả như sau:



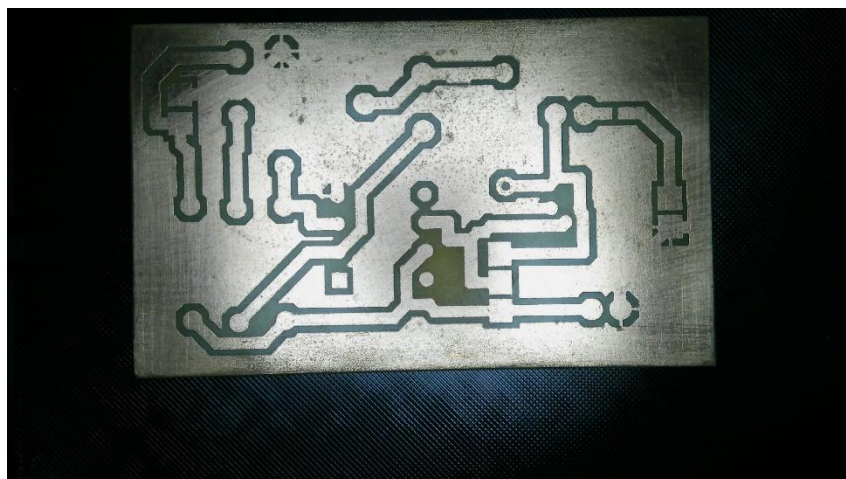
Hình 3.12. Mô hình thực tế

Bên trong mô hình camera được đặt ở trung tâm, và có tấm tản sáng nằm xung quanh. Ở bên dưới lớp tản sáng là cụm đèn tròn led được trang bị:



Hình 3.13. Hình ảnh thực tế vị trí camera

Tấm tản sáng được bố trí để giúp ánh sáng được phân bổ đều, khi ánh sáng được phân bổ được chiếu thẳng vào mạch in sẽ sáng đều không bị hiện tượng sáng chói một phần gây mất chi tiết dẫn đến dự đoán sai. Hình ảnh thực tế khi chụp ảnh mạch in như sau:



Hình 3.14. Ảnh chụp mạch in

CHƯƠNG 4 KẾT QUẢ VÀ THẢO LUẬN

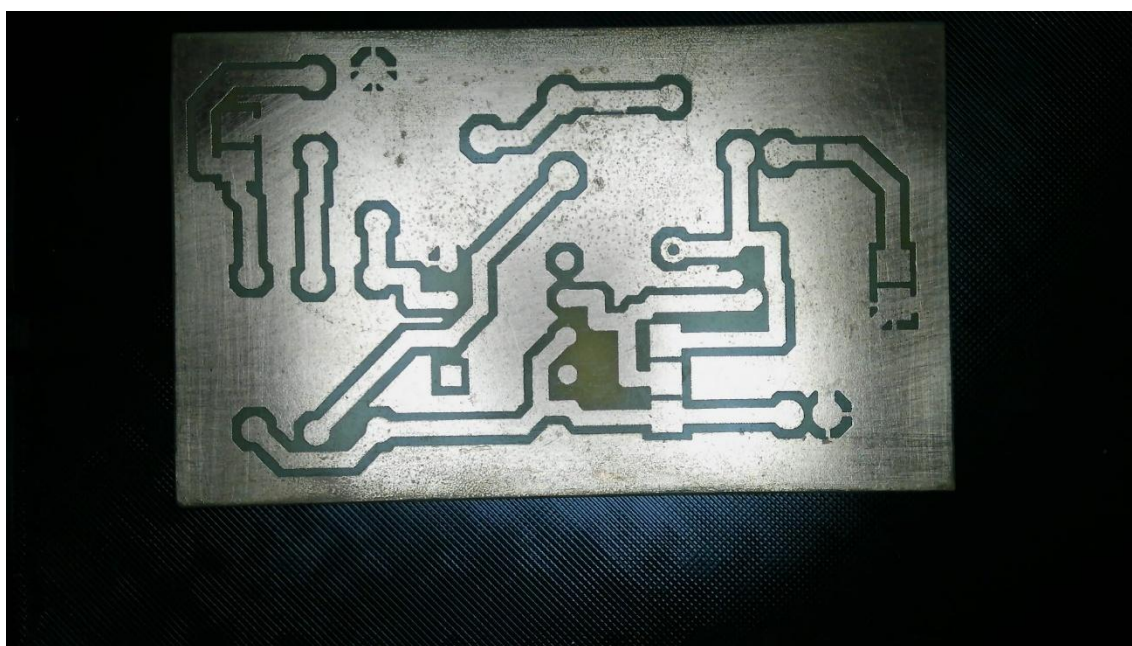
4.1. KẾT QUẢ THIẾT KẾ HỆ THỐNG

Sau khi hoàn thiện mô hình YoLov10s, em đã tiến hành xây dựng một hệ thống cho người dùng có thể đưa mạch PCB vào và chuẩn đoán lỗi. Mô hình được tạo như sau:



Hình 4.1. Sản phẩm thực tế

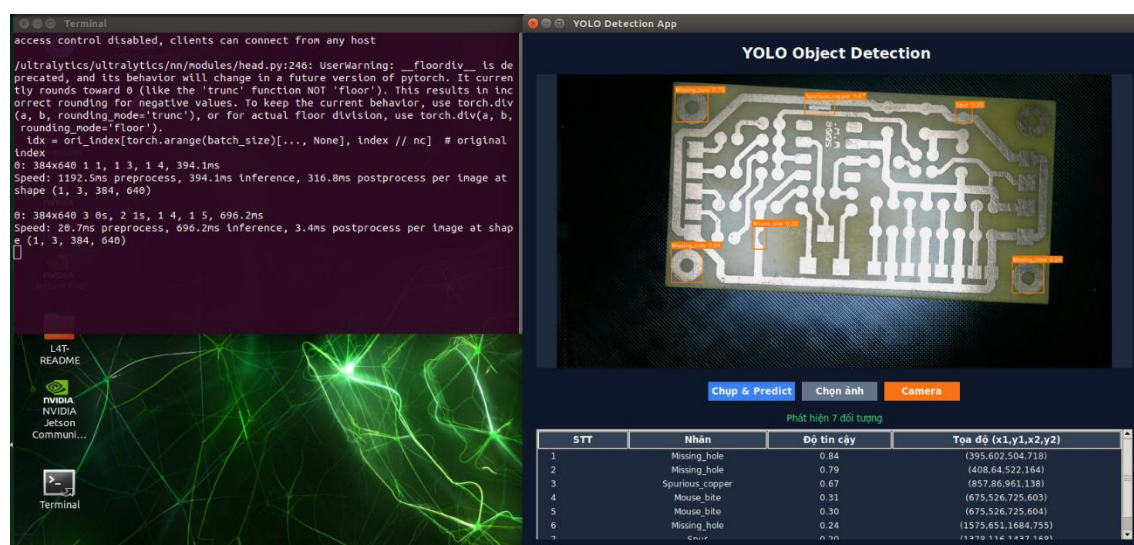
Camera đã được căn chỉnh cho việc lấy nét và bao quát PCB trong khung 10 x 10cm. Hình ảnh khi camera lấy một ảnh PCB như sau:



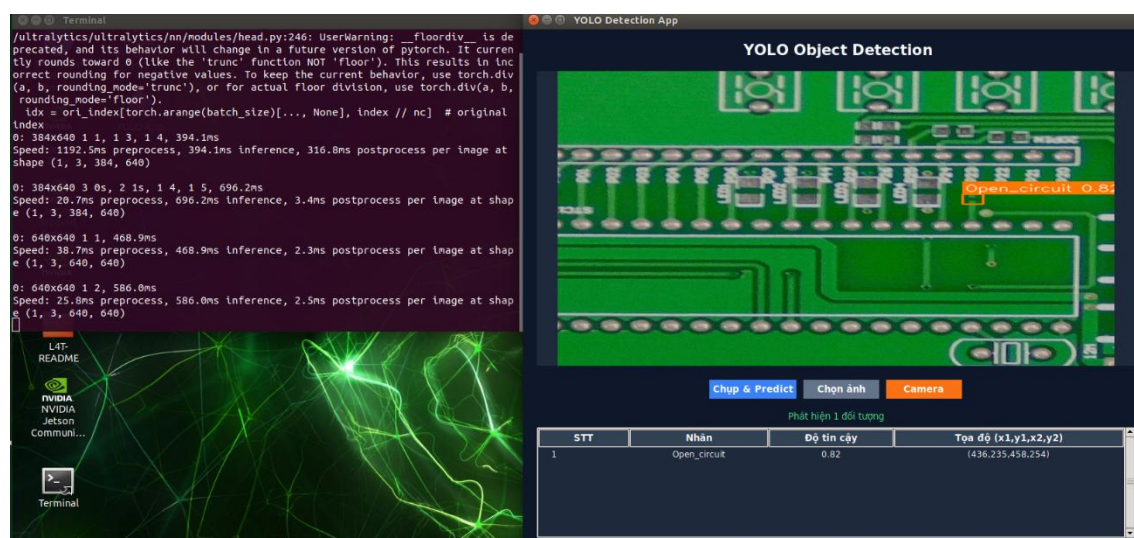
Hình 4.2. Ảnh PCB thông qua camera

Kế tiếp, khởi chạy mô hình, người dùng chỉ cần nhấp chuột 2 lần mở ứng dụng YOLO App thì ứng dụng tự động mở Docker và tự mở giao diện sau đó khởi tạo model yolo. Việc khởi động model mất tầm 2 phút để ứng dụng sẵn sàng hoạt động.

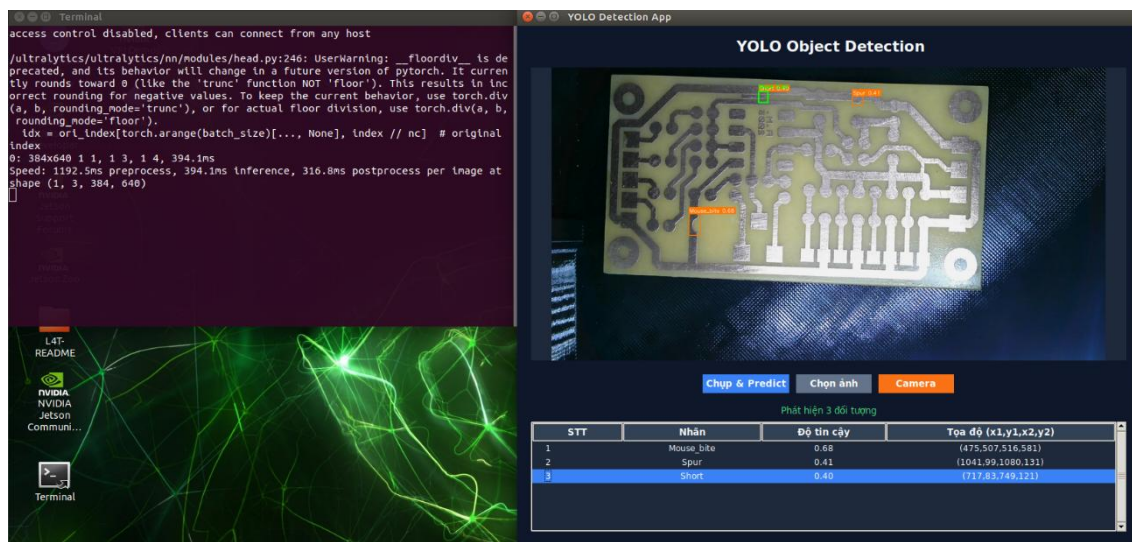
Tiếp theo đó, công việc kiểm thử hiệu suất mô hình qua các mạch PCB thử nghiệm đạt được kết quả như sau:



Hình 4.3. Kết quả mạch thử nghiệm 1



Hình 4.4. Kết quả mạch thử nghiệm 2



Hình 4.5. Kết quả mạch thử nghiệm 3

Về mặt hiệu năng, hệ thống ghi nhận thời gian xử lý trung bình cho mỗi khung hình ở độ phân giải 640 là xấp xỉ 400ms. Tốc độ vận hành này hoàn toàn đáp ứng được các tiêu chí và mục tiêu thiết kế ban đầu đề ra, đồng thời bảo đảm duy trì độ tin cậy cùng tính ổn định cao trong suốt quá trình vận hành liên tục. Song song với tối ưu hóa về mặt thời gian, năng lực nhận diện của hệ thống cũng đạt kết quả rất khả quan. Minh chứng là toàn bộ các lỗi trên mạch in (PCB) đều được phát hiện chính xác và khoanh vùng cục bộ một cách trùng khớp với thực tế. Kết quả nghiệm thu thực nghiệm này khẳng định mô hình đã thực hiện quy trình chẩn đoán lỗi một cách chính xác, chứng minh tính khả thi và hiệu quả cao khi đưa vào ứng dụng thực tiễn.

4.2. ĐÁNH GIÁ MÔ HÌNH

4.2.1. Đánh giá độ chính xác theo từng loại lỗi

Để có cái nhìn chi tiết hơn về hiệu năng thực tế của mô hình, ngoài các chỉ số tổng thể đã trình bày, tôi tiến hành đánh giá riêng biệt khả năng phát hiện từng loại lỗi PCB trên tập kiểm thử thực nghiệm. Kết quả được tổng hợp trong bảng dưới đây:

Bảng 4.1. Kết quả đánh giá độ chính xác theo từng loại lỗi

STT	Loại lỗi	Precision	Recall	Đánh giá
1	Missing hole	0.94	0.92	Rất tốt
2	Mouse bite	0.87	0.83	Tốt
3	Open circuit	0.95	0.94	Rất tốt
4	Short circuit	0.98	0.97	Xuất sắc
5	Spur	0.86	0.85	Tốt
6	Spurious copper	0.96	0.93	Rất tốt

Nhìn chung, mô hình thể hiện hiệu suất nhận diện rất khả quan trên hầu hết các loại lỗi. Lỗi Short circuit đạt kết quả cao nhất với Precision 0.98 và Recall 0.97, điều này có thể lý giải bởi đặc trưng hình học của lỗi này rất rõ ràng — phần đồng dư nối hai đường tín hiệu tạo ra vùng bất thường dễ phân biệt trong không gian đặc trưng của mô hình.

Ngược lại, hai loại lỗi Mouse bite và Spur có kết quả thấp hơn tương đối. Đây là hai loại lỗi có đặc điểm hình học tinh tế: Mouse bite là vết khuyết nhỏ trên mép đường dẫn đồng, còn Spur là gai đồng nhỏ nhô ra nhưng chưa chạm mạch kế bên. Cả hai đều có kích thước pixel rất nhỏ trong ảnh và dễ bị ảnh hưởng bởi điều kiện ánh sáng hoặc nhiễu bề mặt. Tỷ lệ bỏ sót (False Negative) ở mức 13–17% đối với hai loại lỗi này là điểm cần cải thiện trong các giai đoạn phát triển tiếp theo thông qua việc tăng cường dữ liệu huấn luyện hoặc tinh chỉnh ngưỡng tin cậy (confidence threshold) riêng biệt cho từng lớp.

4.2.2 Đánh giá hiệu năng xử lý theo thời gian thực

Bên cạnh độ chính xác nhận diện, tốc độ xử lý là yếu tố sống còn khi triển khai hệ thống lên nền tảng nhúng. Để có đánh giá toàn diện, tôi tiến hành đo lường thời gian xử lý trên nhiều cấu hình đầu vào khác nhau và tổng hợp kết quả như sau:

Bảng 4.2. Thời gian xử lý trung bình theo độ phân giải đầu vào trên Jetson Nano

Độ phân giải ảnh	Thời gian tiền xử lý(ms)	Thời gian suy luận (ms)	Thời gian hậu xử lý(ms)	Tổng (ms)	FPS

640x640	28.7	366.5	4.8	~400	~2-3
320x320	20.5	106.2	3.8	~130	~7-8
160x160	15.3	39.4	3.1	~58	~17

Kết quả đo lường xác nhận rằng ở độ phân giải 640×640 cấu hình được lựa chọn cho hệ thống tổng thời gian xử lý đạt khoảng 400ms, tương đương tốc độ 2–3 FPS. Đây là tốc độ hoàn toàn phù hợp với quy trình kiểm tra bán thủ công mà đề tài hướng tới, trong đó người vận hành chủ động đưa từng bo mạch vào buồng chụp và kích hoạt lệnh nhận diện.

Điểm đáng chú ý là thời gian hậu xử lý của YOLOv10s chỉ khoảng 4.8ms, thấp hơn rất nhiều so với các phiên bản YOLO sử dụng NMS truyền thống (thường từ 8–12ms trên Jetson Nano). Đây là lợi thế trực tiếp từ cơ chế không NMS (NMS-free) của YOLOv10, giúp giải phóng tài nguyên CPU cho các tác vụ song song như duy trì giao diện người dùng và ghi log.

4.2.3. Đánh giá tính ổn định và độ tin cậy vận hành

Ngoài các chỉ số định lượng, hệ thống còn được đánh giá về tính ổn định trong điều kiện vận hành liên tục. Trong quá trình thử nghiệm, hệ thống được chạy liên tục trong khoảng thời gian dài để quan sát các hiện tượng bất thường như tràn bộ nhớ, sụt giảm hiệu năng theo thời gian (thermal throttling) hay lỗi khởi tạo camera.

Kết quả ghi nhận cho thấy hệ thống duy trì hoạt động ổn định, không xuất hiện hiện tượng thoát ứng dụng đột ngột hay tràn bộ nhớ sau nhiều lần thực hiện lệnh nhận diện liên tiếp. Điều này được đảm bảo nhờ:

Thứ nhất, kiến trúc Docker container giúp cô lập hoàn toàn môi trường thực thi, ngăn chặn xung đột thư viện và rò rỉ tài nguyên giữa các phiên làm việc.

Thứ hai, giao diện Tkinter được thiết kế theo mô hình single-thread với vòng lặp sự kiện (event loop) rõ ràng. Khi bắt đầu tác vụ nhận diện, toàn bộ các

nút bấm bị vô hiệu hóa để ngăn người dùng gửi nhiều yêu cầu xử lý đồng thời, từ đó tránh tình trạng quá tải bộ nhớ GPU.

Thứ ba, luồng video từ camera được quản lý tách biệt với luồng suy luận mô hình, đảm bảo việc hiển thị hình ảnh trực tiếp không bị gián đoạn trong quá trình chờ kết quả.

4.2.4. So sánh kết quả với mục tiêu thiết kế ban đầu

Để đánh giá tổng thể mức độ hoàn thành mục tiêu, bảng dưới đây so sánh trực tiếp các chỉ tiêu thiết kế ban đầu với kết quả thực tế đạt được:

Bảng 4.3. So sánh mục tiêu thiết kế và kết quả thực nghiệm

Chỉ tiêu	Mục tiêu thiết kế	Kết quả thực tế	Đánh giá
Thời gian xử lý mỗi ảnh	< 1000 ms	~400 ms	Vượt mục tiêu
Kích thước PCB tối đa	10 × 10 cm	10 × 10 cm	Đạt
Số loại lỗi nhận diện	6 loại	6 loại	Đạt
mAP@0.5	> 0.90	0.9551	Vượt mục tiêu
Giao diện người dùng	Trực quan, dễ dùng	Tkinter, đơn giản	Đạt
Khả năng triển khai	Dễ dàng, đóng gói	Docker container	Đạt
Chi phí phần cứng	Phù hợp SME	~100 USD (Jetson Nano)	Đạt

Kết quả trên cho thấy hệ thống không chỉ đáp ứng mà còn vượt qua các chỉ tiêu thiết kế quan trọng nhất là thời gian xử lý (chỉ bằng 40% hạn mức cho phép) và độ chính xác mAP@0.5 (vượt ngưỡng tối thiểu 0.90). Điều này khẳng định tính khả thi và hiệu quả của giải pháp đề xuất khi áp dụng vào thực tiễn sản xuất tại các doanh nghiệp vừa và nhỏ.

CHƯƠNG 5 TÓM LƯỢC VÀ KẾT LUẬN

5.1 TÓM LƯỢC DỰ ÁN

Đề tài “Xây dựng hệ thống phát hiện khuyết điểm bảng mạch in PCB sử dụng Jetson Nano” được nghiên cứu và triển khai nhằm cung cấp một giải pháp tự động hóa khả thi trong việc nhận diện các lỗi bề mặt phổ biến trên mạch in. Hệ thống hướng tới đối tượng mục tiêu là các doanh nghiệp có quy mô vừa và nhỏ, vốn có nhu cầu tối ưu hóa quy trình kiểm định chất lượng cho các loại PCB kích thước ngắn và vừa với chi phí hợp lý.

Để hiện thực hóa mục tiêu này, dự án đã tập trung nghiên cứu sâu vào việc tích hợp và tối ưu hóa mô hình YOLOv10s trên nền tảng máy tính nhúng NVIDIA Jetson Nano. Quá trình triển khai bao gồm việc huấn luyện mô hình dựa trên tập dữ liệu chuẩn hóa kết hợp với các kỹ thuật tiền xử lý ảnh chuyên sâu, giúp nâng cao tối đa độ chính xác và hiệu năng nhận diện trong điều kiện biên của phần cứng. Song song với đó, giao diện phần mềm dành cho người dùng cũng được nghiên cứu và thiết kế đồng bộ theo hướng tối giản, trực quan và tiết kiệm tài nguyên hệ thống. Giải pháp này không chỉ giảm thiểu gánh nặng tính toán cho thiết bị nhúng mà còn cung cấp đầy đủ các công cụ trực quan hóa giúp người vận hành dễ dàng phát hiện, định vị chính xác vị trí khuyết điểm, từ đó kịp thời đưa ra các phương án khắc phục hiệu quả trong quy trình sản xuất.

5.2 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.2.1 Kết luận

Sau quá trình nghiên cứu lý thuyết, thiết kế hệ thống và triển khai thực nghiệm, đề tài “Xây dựng hệ thống phát hiện khuyết điểm bảng mạch in PCB sử dụng Jetson Nano” đã hoàn thành toàn diện các mục tiêu nghiên cứu được đề ra ban đầu.

Về mặt học thuật và công nghệ, hệ thống đã xây dựng thành công mô hình nhận diện tự động có khả năng phát hiện chính xác 6 loại lỗi phổ biến trên bề mặt mạch in PCB như đã xác định trong mục tiêu đề tài. Kết quả vận hành thực tế cho

thấy mô hình đạt độ chính xác cao, sở hữu độ tin cậy lớn và thể hiện khả năng thích ứng linh hoạt khi thử nghiệm trên nhiều chủng loại bo mạch khác nhau.

Về mặt kỹ thuật và hiệu năng, hệ thống không chỉ đáp ứng mà còn vượt mốc tiêu chí về thời gian xử lý khi chỉ mất xấp xỉ 400ms cho mỗi khung hình, tối ưu hơn đáng kể so với hạn mức 1 giây đặt ra ban đầu. Thiết bị hoạt động ổn định và hiệu quả nhất trong phạm vi kích thước mạch tối đa là 10 x 10 cm. Bên cạnh đó, hệ thống chiếu sáng phụ trợ đi kèm cũng vận hành đạt yêu cầu, cung cấp điều kiện ánh sáng đồng nhất để phục vụ tối ưu cho quá trình thu nhận và xử lý ảnh ngoại vi.

Cuối cùng, phần mềm điều khiển được tối ưu hóa với giao diện thiết kế theo hướng trực quan, đơn giản và thân thiện, giúp người vận hành dễ dàng tiếp cận, làm chủ công nghệ mà không đòi hỏi quy trình đào tạo phức tạp.

5.2.2 Hạn chế và hướng phát triển

Mặc dù đã đạt được những kết quả khả quan ban đầu, hệ thống vẫn tồn tại một số điểm hạn chế nhất định cần được nhìn nhận và khắc phục trong các giai đoạn phát triển tiếp theo:

- Giới hạn về năng lực xử lý của phần cứng: Nền tảng máy tính nhúng NVIDIA Jetson Nano sở hữu cấu hình GPU ở mức cơ bản, dẫn đến thời gian phản hồi trung bình cho mỗi ảnh còn duy trì ở mức xấp xỉ 400ms. Tốc độ này tuy đạt yêu cầu thiết kế ban đầu nhưng sẽ gặp thách thức lớn khi triển khai vào môi trường sản xuất của các doanh nghiệp quy mô lớn, nơi các băng chuyền công nghiệp đòi hỏi tốc độ kiểm định liên tục và thời gian thực (real-time) khắt khe hơn.
- Trải nghiệm người dùng và tính thẩm mỹ của giao diện: Giao diện đồ họa hiện tại mới chỉ tập trung vào việc đáp ứng các tính năng cốt lõi và vận hành theo lối thiết kế truyền thống. Hệ thống chưa thực sự tối ưu hóa về mặt thẩm mỹ cũng như trải nghiệm người dùng (UI/UX) theo các tiêu chuẩn thiết kế hiện đại, điều này có thể ảnh hưởng đến mức độ trực quan khi vận hành lâu dài. Hạn chế trong việc xử lý bo mạch kích thước lớn: Đối với các dòng

bảng mạch có kích cỡ lớn hoặc mật độ linh kiện cao, việc phân tích trên toàn bộ khung hình ảnh gốc dễ làm giảm độ chính xác của mô hình.

- Hệ thống hiện tại chưa tích hợp giải thuật phân tách ảnh (tách khối ảnh lớn thành các phân vùng nhỏ hơn trước khi đưa vào mô hình nhận diện), điều này làm giới hạn khả năng mở rộng phạm vi kiểm định đối với các sản phẩm PCB phức tạp.
- Quản trị và lưu trữ cơ sở dữ liệu: Hệ thống hiện vẫn thiếu đi một cơ chế quản lý dữ liệu tập trung. Việc chưa thể tự động lưu trữ, phân loại hình ảnh mạch lỗi và thống kê lịch sử kiểm định một cách hệ thống làm hạn chế khả năng khai thác dữ liệu của doanh nghiệp, gây khó khăn cho việc phân tích nguyên nhân lỗi để cải tiến quy trình sản xuất.

Từ những kết quả đã đạt được cùng các điểm hạn chế còn tồn tại, đề tài định hướng các nghiên cứu và cải tiến tiếp theo tập trung vào các giải pháp trọng tâm sau:

Nâng cấp nền tảng phần cứng công nghiệp: Tiến hành chuyển đổi và triển khai mô hình lên các dòng máy tính nhúng có hiệu năng xử lý cao hơn thuộc hệ sinh thái NVIDIA Jetson (như Jetson Orin Nano hoặc Jetson Xavier NX). Giải pháp này nhằm tối ưu hóa đáng kể tốc độ suy luận của mô hình, hướng tới việc đáp ứng hoàn hảo yêu cầu kiểm định thời gian thực trên các băng chuyền sản xuất tự động tốc độ cao tại các nhà máy quy mô lớn.

Tối ưu hóa thuật toán cho bo mạch kích thước lớn: Nghiên cứu và tích hợp kỹ thuật phân mảnh hình ảnh (tách ảnh độ phân giải cao thành các vùng lưới nhỏ hơn - SAHI) trước khi đưa vào mạng nơ-ron. Phương pháp này sẽ giúp mô hình duy trì và nâng cao độ chính xác khi nhận diện các khiếm khuyết có kích thước siêu nhỏ trên các dòng PCB khổ lớn hoặc có mật độ vi mạch dày đặc.

Hiện đại hóa giao diện và trải nghiệm người dùng: Thiết kế lại toàn bộ giao diện đồ họa (GUI) dựa trên các tiêu chuẩn UI/UX hiện đại. Mục tiêu là hướng tới sự trực quan, chuyên nghiệp, hỗ trợ đa nền tảng và tích hợp các công cụ cảnh báo

thông minh, giúp người vận hành dễ dàng tương tác và giám sát hệ thống trong thời gian dài.

Xây dựng hệ thống quản trị dữ liệu và phân tích thống kê: Phát triển một cơ sở dữ liệu tập trung nhằm tự động lưu trữ hình ảnh mạch lỗi, phân loại chủng loại khuyết điểm và thời gian ghi nhận. Từ nguồn dữ liệu này, hệ thống sẽ xây dựng các biểu đồ báo cáo thống kê trực quan, hỗ trợ bộ phận quản lý chất lượng (QA/QC) dễ dàng đánh giá xu hướng lỗi, từ đó có cơ sở khoa học để cải tiến quy trình chế tạo mạch in của doanh nghiệp.

Ngoài các định hướng phát triển đã nêu, đề tài còn ghi nhận một số hướng mở rộng tiềm năng khác có thể được khai thác trong các nghiên cứu tương lai:

Tích hợp kỹ thuật lượng tử hóa mô hình (Model Quantization): Hiện tại mô hình được triển khai ở định dạng PyTorch FP32 thông thường. Việc chuyển đổi sang định dạng TensorRT INT8 hoặc FP16 có thể giảm thời gian suy luận thêm 30–50% mà không làm suy giảm đáng kể độ chính xác. Đây là bước tối ưu hóa tiếp theo rất khả thi trên Jetson Nano do thiết bị đã hỗ trợ sẵn TensorRT trong JetPack SDK.

Mở rộng tập dữ liệu huấn luyện từ ảnh thực tế thu thập tại cơ sở sản xuất: Tập dữ liệu hiện tại được lấy từ Kaggle với ảnh PCB tổng hợp. Trong thực tế, các bo mạch từ các nhà sản xuất khác nhau có màu sắc, chất liệu và mật độ linh kiện khác nhau, dẫn đến sự phân phối dữ liệu (domain shift) có thể ảnh hưởng đến độ chính xác. Việc thu thập và bổ sung ảnh từ chính dây chuyền sản xuất của doanh nghiệp ứng dụng vào tập huấn luyện sẽ cải thiện đáng kể hiệu năng thực tế.

Phát triển theo hướng tự động hóa hoàn toàn (Full Automation): Phiên bản hiện tại yêu cầu người vận hành thủ công đưa bo mạch vào buồng kiểm tra và nhấn nút kích hoạt. Trong tương lai, có thể tích hợp cơ cấu truyền động (băng tải mini) và cảm biến quang điện để tự động phát hiện khi có bo mạch được đặt vào và kích

hoạt lệnh nhận diện mà không cần can thiệp của con người, nâng cấp hệ thống từ bán tự động lên hoàn toàn tự động.

Xây dựng giao thức kết nối với hệ thống quản lý sản xuất (MES): Trong môi trường nhà máy thông minh, hệ thống kiểm tra chất lượng cần có khả năng giao tiếp với các hệ thống quản lý sản xuất cấp cao hơn. Việc phát triển API REST hoặc giao thức MQTT để truyền kết quả kiểm tra theo thời gian thực lên hệ thống MES sẽ giúp doanh nghiệp theo dõi tỉ lệ lỗi theo lô sản xuất, phân tích xu hướng và đưa ra cảnh báo sớm khi tỉ lệ lỗi vượt ngưỡng cho phép.

TÀI LIỆU THAM KHẢO

- [1] K. Schwab, The Fourth Industrial Revolution, Switzerland: World Economic Forum, 2016.
- [2] M. & Co. Company, "Industry 4.0: Capturing value at scale in discrete manufacturing," 2019.
- [3] M. Arora, "AI-Driven Industry 4.0: Advancing Quality Control through Cutting-Edge Image Processing for Automated Defect Detection," *A Monthly Journal of Computer Science and Information Technology*, vol. 12, no. 8, pp. 16-32, 2023.
- [4] C. S. T. A. Tushar Prasanna Swaminathan, "BENCHMARKING DEEP LEARNING MODELS ON NVIDIA JETSON NANO FOR REAL-TIME SYSTEMS: AN EMPIRICAL INVESTIGATION," p. 26, 2024.
- [5] A. M. Q. M. u. H. M. E. I. Wahyu Rahmانيar, "AI in Industry: Real-World Applications and Case Studies," *AI IN INDUSTRY*, 2023.
- [6] Q. L. a. N. A. M. ISA, "Printed Circuit Board Defect Detection Methods Based on Image Processing, Machine Learning and Deep Learning: A Survey," IEEE, 2023.
- [7] V. FMIT, "FMIT INSTITUTE," [Online]. Available: [https://fmit.vn/en/glossary/object-detection-label#:~:text=Object%20Detection%20\(ph%C3%A1t%20hi%E1%BB%87n%20%C4%91%E1%BB%91i,%E2%80%93%20bounding%20box\)%%20c%E1%BB%A7a%20ch%C3%BAng..](https://fmit.vn/en/glossary/object-detection-label#:~:text=Object%20Detection%20(ph%C3%A1t%20hi%E1%BB%87n%20%C4%91%E1%BB%91i,%E2%80%93%20bounding%20box)%%20c%E1%BB%A7a%20ch%C3%BAng..) [Accessed 26 March 2026].
- [8] P. P., "What is YOLOv10? An Architecture Deep Dive," 14 June 2024. [Online]. Available: <https://blog.roboflow.com/what-is-yolov10/>. [Accessed 3 March 2026].
- [9] Ultralytics, "YOLOv10: Real-Time End-to-End Object Detection," [Online]. Available: <https://docs.ultralytics.com/models/yolov10/#what-are->

- the-performance-benchmarks-for-yolov10-models. [Accessed 26 March 2026].
- [10] J. Wang, "Video Analytics Solution in Airports for Operation Efficiency Improving and Safety Enhancement," 2023. [Online]. Available: <https://www.seeedstudio.com/blog/2023/06/29/video-analytics-solution-in-airports-for-operation-efficiency-improving-and-safety-enhancement/>. [Accessed 27 March 2026].
- [11] 2. L. Education, "Docker là gì ? Kiến thức cơ bản về Docker," 14 February 2020. [Online]. Available: <https://viblo.asia/p/docker-la-gi-kien-thuc-co-ban-ve-docker-maGK7qeelj2>. [Accessed 27 March 2026].
- [12] H. L. K. Z. L. J. H. G. D. Ao Wang, "YOLOv10: Real-Time End-to-End Object Detection," p. 8, 30 October 2024.
- [13] "Ultralytics YOLO26," Ultralytics, 14 January 2026. [Online]. Available: <https://docs.ultralytics.com/vi/models/yolo26>. [Accessed 21 May 2026].